

BMM Developer Notes

Table of contents

Overview	5
1 System setup	8
1.1 Setting up a local fork of the bmm repository	8
1.2 Prerequisites	8
1.3 Package development via RStudio and devtools	9
1.4 The local/ directory	10
2 Git and Github workflow	11
2.1 Git commands	11
2.2 Pull requests	12
3 BMM code structure	13
3.1 The main workhorse - <code>bmm()</code>	13
3.2 The <code>bmmformula</code> class	15
3.3 Models	16
3.4 Model domains and class chains	19
3.5 S3 methods	19
3.6 Data flow: the attribute bridge pattern	21
3.7 Default priors structure	22
3.8 Suppressing mu for models without a location parameter	22
3.9 The <code>init_ranges</code> field and <code>create_initfun()</code>	23
3.10 Within-chain threading	24
3.11 Version tables	25
M3 nests the table one level deeper	26
Not every model uses either pattern	27
3.12 File organization	27
R/bmm.R	27
R/helpers-*.R	28
R/bmmformula.R	28
R/model_*.R	29
R/distributions.R	29
R/conditional_effects.R, R/emmeans.R, R/pp_check.R	29
R/utils.R, R/brms-misc.R, R/restructure.R, R/summary.R, R/update.R	30
inst/stan_chunks/	30

4	Example model file	31
4.1	The Interference Measurement Model (IMM)	31
4.1.1	Model definition	31
4.1.2	Model alias	34
4.1.3	check_data() methods	37
4.1.4	configure_model() methods	39
4.2	The Signal Discrimination Model (SDM)	41
4.2.1	Model definition	41
4.2.2	check_data() methods	43
4.2.3	configure_model() methods	44
4.2.4	Post-processing methods	46
4.3	The Diffusion Decision Model (DDM)	47
4.3.1	Model definition	47
4.3.2	User-facing alias	49
4.3.3	bmf2bf method	49
4.3.4	configure_model method	49
4.3.5	check_data method	50
4.4	The EZ-Diffusion Model (EZDM)	51
4.4.1	Version table	52
4.4.2	bmf2bf for aggregate data	53
4.4.3	Version-specific configure_model	54
5	Adding a new model	55
	What use_model_template() does	55
5.1	Example	56
5.2	Failure modes to expect	62
5.3	Testing	63
5.3.1	Parameter recovery	64
5.3.2	Distribution functions	65
5.4	Post-processing infrastructure	65
5.5	Add an example dataset	65
5.6	Add an article	66
6	Coding conventions	67
6.1	Errors, warnings and messages	67
6.2	Dependencies	68
6.3	Code shape	69
6.4	S3 conventions	69
6.5	Where to validate arguments	70
6.6	log_lik and posterior_predict	70
6.7	Naming traps	71
6.8	Documentation and the checks that enforce it	72

6.9	Tests	73
6.9.1	Parameter recovery, and the circularity trap	73

Overview

Last update: 16.09.2026

This guide aims to help developers contribute new models to `bmm`. It is a work in progress and will be updated as the package evolves. It explains how to set-up your system for package development, the structure of the package, and the workflow for contributing new models to the package.

The `bmm` package now supports models across multiple domains, including visual working memory models (mixture models, IMM, SDM), response time / decision models (DDM, EZDM, Censored-Shifted Wald), and categorical response models (M3). The package requires **R** $\geq 4.1.0$ and uses the native pipe operator `|>` throughout. Further models are in development on their own feature branches (LBA, the log-normal race model, and a utility model, among others); they are not part of a release and not covered here until they are.

This guide describes `bmm` at the **v1.3.2** tag. Anything merged after that tag may not be reflected here yet, and the version on CRAN can lag behind the tag while a submission is being processed. If you run into problems, don't hesitate to [open an issue on github](#).

We follow a [github flow workflow](#). The repository contains two main branches:

- Master (contains the latest released stable version of the `bmm` package)
- Develop (contains the latest stable development branch)

All new feature development should occur on an independent branch from Develop. If you want to contribute a new model to the `bmm` package, you need to fork the repository, create a new branch for your model off Develop, extensively test the model, and eventually submit a pull request into the Develop branch of the main repository.

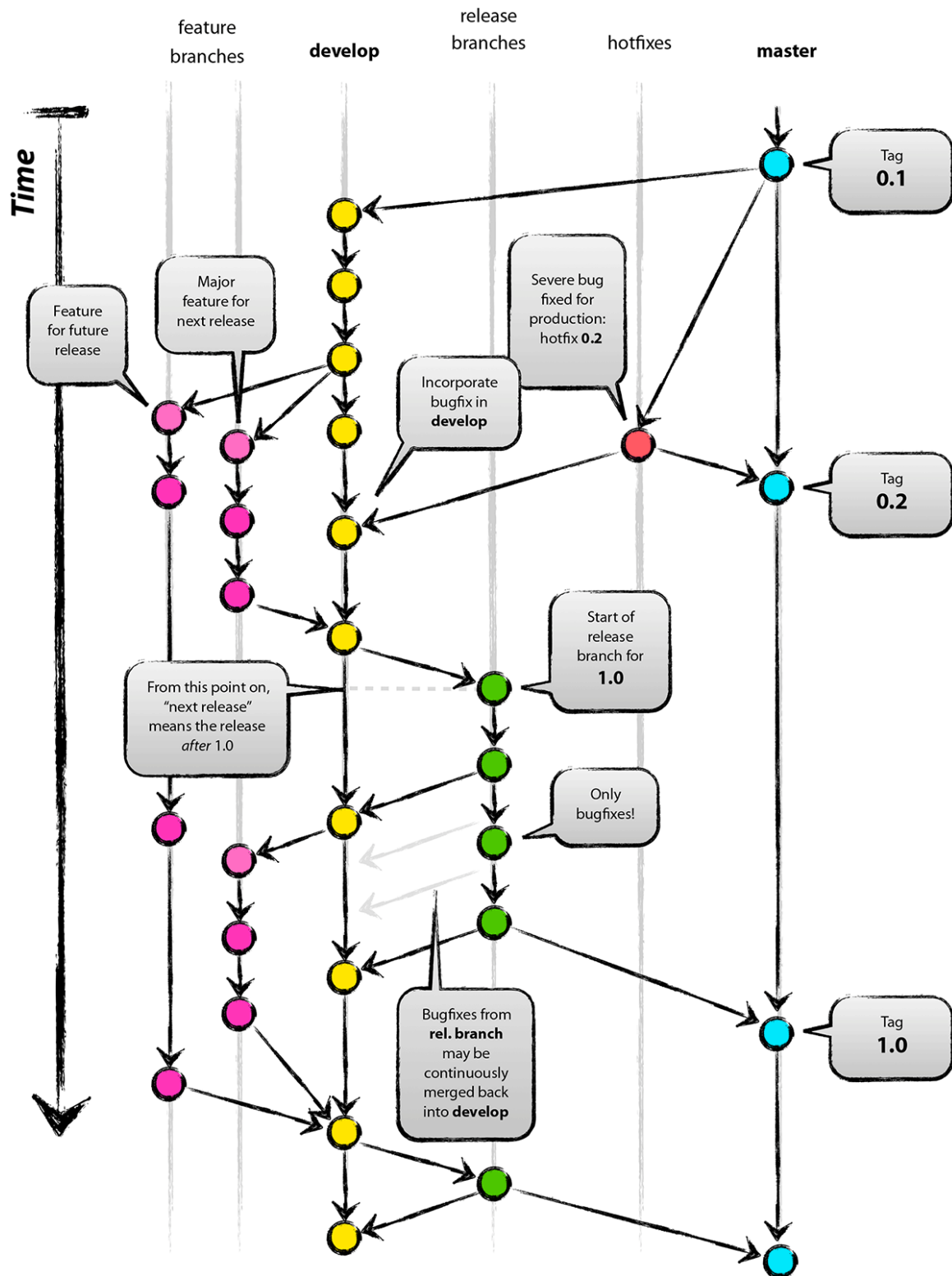
How that pull request is reviewed:

- One reviewer approval is required to merge. Two are required when the pull request touches the analysis pipeline files or the exported API, because those changes affect every model rather than only yours.
- An automated AI functionality review runs on every pull request as a required status check. It reads the diff and reports what it thinks the code does and where it disagrees with the description. It never counts as an approval, and a human still reviews the same code.

- A pull request that adds a new model is approved by a human who has installed your branch and fitted the model. Reading the diff is not sufficient for a new model, because the failure modes that matter (a likelihood that compiles but samples badly, priors that make the model unidentifiable, initial values that hit the boundary) only show up in a fit.

Accordingly, make the model easy to fit for whoever reviews it: include a short, runnable example with a small simulated dataset, and say in the pull request how long it takes to fit and what output to expect.

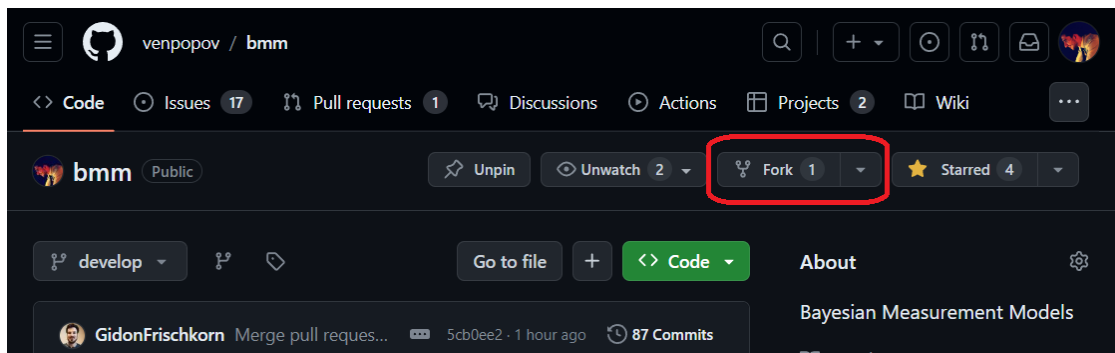
Once your changes are merged into the Develop branch, they will be included in the next release of the package.



1 System setup

1.1 Setting up a local fork of the bmm repository

1. Fork the bmm github repository. This will create a copy of the current development branch into your own github account



2. [Clone](#) your fork to your local machine
3. [Create a new branch](#) for your model off `develop`. Open an issue first and name the branch after it, as `<issue-number>-short-description` (for example `349-lba-model`). That is what the branches in the main repository look like, and it makes the branch, the issue and the eventual pull request easy to line up.

1.2 Prerequisites

The bmm package requires **R** $\geq 4.1.0$ (for native pipe `|>` support). The package works with both `rstan` and `cmdstanr` backends. `cmdstanr` is generally recommended for faster compilation and better error messages. You can install it with:

```
install.packages("cmdstanr", repos = c("https://stan-dev.r-universe.dev", getOption("repos")))
cmdstanr::install_cmdstan()
```

1.3 Package development via RStudio and devtools

The `bmm` package is setup as an RStudio project. Opening the `bmm.Rproj` file will open a new RStudio instance, which facilitates package development with a few commands from the `devtools` package. A great tutorial on package development can be found [here](#). Below is a summary of the most important steps

1. Make sure you have the `devtools` package and a few others installed and loaded

```
install.packages(c("devtools", "roxygen2", "testthat", "knitr"))
library(devtools)
install_dev_deps()
```

To avoid having to load the **devtools** package every time, you can add the following code to your `.Rprofile` file

```
if (interactive()) {
  suppressMessages(require(devtools))
}
```

As noted [here](#), you can create and open an `.Rprofile` file, if you don't already have one with

```
use_devtools()
```

2. Load the current version of the `bmm` package based on your local files

```
load_all() # or ctrl+shift+L
```

you can use this command whenever you make changes to the package code to see the changes in action. You should not call `library(bmm)` or source the files manually, as this will load the installed version of the package, not the one you are developing.

3. Make any changes to the package code that you need to make (elaborated in the next section)
4. Use `check()` to check the package for errors and warnings

```
check()
```

you should always ensure that `check()` produces no errors before submitting a pull request

5. Use `document()` to update the documentation

```
document()
```

1.4 The `local/` directory

The `local/` directory in the `bmm` repository is used for machine-specific scripts that are not tracked by `git`. When developing a new model, you should create a parameter recovery script at `local/<model_name>_parameter_recovery.R`. This script should simulate data from the model, fit it, and verify that parameters are recovered accurately. These scripts are not run by `R CMD check` but are expected for all new models.

2 Git and Github workflow

(This is not meant to be a complete guide to git, but rather a short summary of key commands)

After your fork the `bmm` repository and create a branch for your new model/feature, you can follow a typical git workflow. As you make changes and add new files, you will want to:

2.1 Git commands

[Add](#) your changes to git, [commit](#) them and then [push](#) your changes to your forked repository. You can run these commands from a terminal or from the RStudio terminal (with the project working directory)

Add changed files to the staging area. It is best to add specific files rather than using `git add *` or `git add .`, which can accidentally stage unintended files (e.g., `.env`, large data files, or files in `local/`):

```
git add R/model_my_model.R inst/stan_chunks/my_model_funs.stan
```

Commit the changes to the local repository

```
git commit -m "A short message describing the changes you made"
```

Push the changes to your forked repository

```
git push
```

You can (and should) repeat this process as many times as you need to before submitting a pull request. This will allow you to make many small changes and test them before submitting a pull request. Ideally each commit should be a small, self-contained change that can be easily reviewed.

2.2 Pull requests

When you are ready you can open a pull request from your forked repository to the main bmm repository. You can do this from the github website. Make sure to select the Develop branch as the base branch and your feature branch as the compare branch. You should add a detailed description of your changes, including the motivation for the changes and any relevant context. You should also mention any issues that your pull request resolves.

3 BMM code structure

Adding a new model is straightforward using the `use_model_template()` function, which will be described in the next section. You do not have to edit any of the files below, but it will be helpful to understand the structure of the package.

3.1 The main workhorse - `bmm()`

The main function for fitting models is `bmm()`. This function is the main entry point for users to fit models. It is set-up to be independent of the specific models that are implemented in the package.

```
bmm <- function(formula, data, model,
                prior = NULL,
                sort_data = getOption("bmm.sort_data", "check"),
                silent = getOption("bmm.silent", 1),
                backend = getOption("brms.backend", NULL),
                file = NULL, file_compress = TRUE,
                file_refit = getOption("bmm.file_refit", FALSE), ...) {
  deprecated_args(...)
  dots <- list(...)
  local_brms_threads(dots)

  # check if the model has been previously fit and return it if requested
  x <- try_read_bmmfit(file, file_refit)
  if (!is.null(x)) {
    return(x)
  }

  # set temporary global options and return modified arguments for brms
  configure_opts <- nlist(
    sort_data, silent, backend,
    parallel = dots$parallel,
    cores = dots$cores
  )
}
```

```

opts <- configure_options(configure_opts)
dots$parallel <- NULL

# check model, formula and data, and transform data if necessary
user_formula <- formula
model <- check_model(model, data, formula)
data <- check_data(model, data, formula)
formula <- check_formula(model, data, formula)

# generate the model specification to pass to brms later
config_args <- configure_model(model, data, formula)

# configure the default prior and combine with user-specified prior
prior <- configure_prior(model, data, config_args$formula, prior)

# configure initial values if necessary
config_args$init <- create_initfun(model, data, config_args$formula)

# estimate the model
fit_args <- combine_args(nlist(config_args, opts, dots, prior))
fit <- brms::do_call(brms::brm, fit_args)

# model post-processing
fit <- postprocess_brm(
  model, fit,
  fit_args = fit_args,
  user_formula = user_formula,
  configure_opts = configure_opts
)

# save the fitted model object if file argument provided and return the object
try_save_bmmfit(fit, file, compress = file_compress)
}

```

It calls several subroutines, implemented as generic S3 methods, to:

- `configure_options()` - configure local options for fitting, such as parallel sampling
- `check_model()` - check if the model exists and is valid
- `check_data()` - check whether the data contains all necessary information and transform it
- `check_formula()` - check if the formula is specified correctly

- `configure_model(model, data, formula)` - configure the model specification (formula, family, Stan code) to pass to `brms`
- `configure_prior()` - set the default priors for the model and combine them with the user prior
- `create_initfun()` - create an initial values function for MCMC sampling
- `postprocess_brm()` - post-process the fitted model (e.g., adjust link functions, add metadata)

In addition, it also tests if the specified `bmmmodel` has already been estimated and saved to a file. This is done via the `try_read_bmmfit` and `try_save_bmmfit` functions.

The first line worth a second look is `local_brms_threads(dots)`. `threads` is a `brms::brm()` argument and reaches it through `...` like any other, so `bmm()` does not need to know about it — except that a model may need to *generate different Stan code* when threading is on. `local_brms_threads()` (R/utils.R:128) normalizes whatever the user passed (a bare number becomes `brms::threading(n)`) and sets the `brms.threads` option for the duration of the call, so that a `configure_model()` method further down the pipeline can read it. Only `sdm` does this today; see [Within-chain threading](#).

3.2 The `bmmformula` class

Before understanding models, it's important to understand how users specify what to estimate. The `bmmformula` (alias: `bmf()`) is a list of formulas, one per model parameter. Unlike `brmsformula`, there is no response variable on the left-hand side — the response is specified when constructing the model object.

```
# Each formula predicts one model parameter
f <- bmmformula(
  kappa ~ 0 + set_size + (1 | id),
  c ~ 1 + (1 | id),
  a ~ 1
)

# Equivalent shorthand
f <- bmf(kappa ~ 0 + set_size + (1 | id), c ~ 1 + (1 | id), a ~ 1)
```

Parameters can be **fixed to a constant** using `=` instead of `~`:

```
f <- bmf(kappa ~ condition, a = 0.5) # 'a' is fixed to 0.5, not estimated
```

Parameters **not mentioned** in the formula are automatically given an intercept-only formula (`param ~ 1`) and a message is printed.

The `bmmformula` is converted to a `brmsformula` during model configuration via the `bmf2bf()` S3 method. This conversion adds the response variable as the first formula line and translates each parameter formula into either `brms::lf()` (linear) or `brms::nlf()` (non-linear) components.

3.3 Models

All models in the package are defined as S3 classes and follow a strict template. This allows us to implement general methods for handling model fitting, data checking, and post-processing. Each model has an internal constructor (`.model_<name>()`) that defines the model and its parameters, and a user-facing alias. To keep the specification easy to read and edit, the parameters, links, fixed parameters, priors, and initialization ranges live in a separate `.<name>_defaults` list — or, for models with several versions, in a `.<name>_version_table` (see [Version tables](#)) — and the constructor spells out each field of the model object inline, reading from that list. For a complete example model file and an explanation, see [Section 4](#). The general model template looks like this:

```
# the model's parameter specification (one such list per version in a version table)
.my_new_model_defaults <- list(
  parameters = list(
    par1 = "Parameter 1 = description of parameter 1",
    par2 = "Parameter 2 = description of parameter 2"
  ),
  links = list(par1 = "identity", par2 = "log"),
  fixed_parameters = list(mu = 0),
  priors = list(
    par1 = list(main = "normal(0, 1)", effects = "normal(0, 0.5)"),
    par2 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.5)")
  ),
  init_ranges = list(par1 = c(-1, 1), par2 = c(0.5, 1.5))
)

.model_my_new_model <- function(resp_var1 = NULL, required_arg1 = NULL,
                                required_arg2 = NULL, links = NULL,
                                call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_var1),
      other_vars = nlist(required_arg1, required_arg2),
      domain = "",
```

```

    task = "",
    name = "",
    citation = "",
    version = "NA",
    requirements = "",
    parameters = .my_new_model_defaults[["parameters"]],
    links = .my_new_model_defaults[["links"]],
    fixed_parameters = .my_new_model_defaults[["fixed_parameters"]],
    default_priors = .my_new_model_defaults[["priors"]],
    init_ranges = .my_new_model_defaults[["init_ranges"]]
  ),
  class = c("bmmmodel", "my_new_model"),
  call = call
)
out$links[names(links)] <- links
out
}

```

The `.my_new_model_defaults` list holds the parameter specification (one such list per version in a version table). Its fields are:

- **parameters** - Named list of model parameters with descriptions
- **links** - Named list of link functions for each parameter. The available options are "identity", "log", "softplus", "log1p", "logm1", "inverse", "sqrt", "logit", "probit", "tan_half" and "cloglog" (R/helpers-parameters.R:76). "softplus", added in 1.3.2 (#363), is an opt-in alternative to "log" for positively-bounded parameters: it keeps the parameter positive but grows linearly rather than exponentially, so predictor effects are additive on the natural scale and large values do not blow up. Users can override any of these per parameter through the model alias's `links` argument
- **fixed_parameters** - Named list of parameters fixed to constant values (e.g., `list(mu = 0)`)
- **priors** - Named list with **main** (intercept) and **effects** (slopes) priors per parameter. The constructor stores this under the model object's `default_priors` field
- **init_ranges** - Named list of `c(lower, upper)` vectors for MCMC initialization (see Section 3.9)

The constructor copies these into the model object (renaming `priors` to `default_priors`) alongside the response and auxiliary variables (`resp_vars`, `other_vars`), the model metadata (`domain`, `task`, `name`, `citation`, `requirements`), and the `version`. Keeping the fields inline means a new field on the model object is a one-line addition you can see and copy directly; the trade-off is that older models need updating when a field is added (a periodic, mechanical refactor).

Each model is accompanied by a user-facing alias, the documentation of which is generated automatically based on the info list in the model definition.

```
# user facing alias
# information in the title and details sections will be filled in
# automatically based on the information in the .model_modelname()$info
#' @title `r .model_my_new_model()$name`
#' @name my_new_model
#' @details `r model_info(.model_my_new_model())`
#' @param resp_var1 A description of the response variable
#' @param required_arg1 A description of the required argument
#' @param required_arg2 A description of the required argument
#' @param ... used internally for testing, ignore it
#' @return An object of class `bmmmodel`
#' @export
#' @examples
#' \dontrun{
#' # put a full example here (see 'R/model_ddm.R' for an example)
#' }
my_new_model <- function(resp_var1, required_arg1, required_arg2,
                          links = NULL, ...) {
  call <- match.call()
  stop_missing_args()
  .model_my_new_model(resp_var1 = resp_var1, required_arg1 = required_arg1,
                      required_arg2 = required_arg2, links = links,
                      call = call, ...)
}
```

For a versioned model, the alias instead carries a `version = c("v1", "v2")` argument validated with `version <- match.arg(version)`, and passes the chosen `version` on to the constructor (see `cswald` for a worked example). `use_model_template()` generates whichever variant you ask for via its `versions` argument.

Then users can fit the model using the `bmm()` function, and the model will be automatically recognized and handled by the package:

```
fit <- bmm(formula = my_bmmformula,
           data = my_data,
           model = my_new_model(resp_var1, required_arg1, required_arg2))
```

3.4 Model domains and class chains

Models use hierarchical S3 class chains that encode the model taxonomy. The class chain determines which S3 methods are dispatched. The general pattern is:

```
class = c("bmmodel", <domain/intermediate>, <model>, <version>)
```

The current model families and their class chains are:

Circular / Visual working memory models:

- mixture2p: `c("bmmodel", "circular", "mixture2p")`
- mixture3p: `c("bmmodel", "circular", "non_targets", "mixture3p")`
- imm (full): `c("bmmodel", "circular", "non_targets", "imm", "imm_full")`
- sdm: `c("bmmodel", "circular", "sdm", "sdm_simple")`

Response time / Decision models:

- ddm: `c("bmmodel", "ddm")`
- ezdm (3par): `c("bmmodel", "ezdm", "ezdm_3par")`
- cswald (simple): `c("bmmodel", "cswald", "cswald_simple")`

Categorical response models:

- m3 (ss): `c("bmmodel", "m3", "m3_ss")`

Intermediate classes like "circular" and "non_targets" allow shared `check_data` methods (e.g., `check_data.circular` validates that responses are in radians, `check_data.non_targets` validates non-target feature columns). S3 methods chain via `NextMethod()` from specific to general.

3.5 S3 methods

The package uses S3 methods to handle different models. This means that the same function can behave differently depending on the class of the object it is called with. For example, the `configure_model(model, data, formula)` function called by `bmm()`, is generally defined as:

```
configure_model <- function(model, data, formula) {  
  UseMethod('configure_model')  
}
```

and it will call a function `configure_model.modelname()` that is specified for each model. The same is true for other functions, such as `check_data()`, `check_formula()`, `configure_prior()`, `create_initfun()`, and `postprocess_brm()`. This allows us to add new models without having to edit the main fitting function, `bmm()`.

The key S3 generics in the pipeline are:

Generic	Signature	Defined in
<code>check_model</code>	<code>(model, data = NULL, formula = NULL)</code>	<code>helpers-model.R</code>
<code>check_data</code>	<code>(model, data, formula)</code>	<code>helpers-data.R</code>
<code>check_formula</code>	<code>(model, data, formula)</code>	<code>bmmformula.R</code>
<code>configure_model</code>	<code>(model, data, formula)</code>	<code>helpers-model.R</code>
<code>configure_prior</code>	<code>(model, data, formula, user_prior, ...)</code>	<code>helpers-prior.R</code>
<code>create_initfun</code>	<code>(model, data, formula)</code>	<code>helpers-inits.R</code>
<code>postprocess_brm</code>	<code>(model, fit, ...)</code>	<code>helpers-postprocess.R</code>
<code>bmf2bf</code>	<code>(model, formula)</code>	<code>bmmformula.R</code>

Match these signatures exactly in your methods, including the `...` where the generic has one and the `= NULL` defaults on `check_model`. R CMD `check` compares every method against its generic, and a method that drops an argument the generic declares is reported under “checking S3 generic/method consistency”. Every method in the package follows this: all four `configure_prior.*` methods carry `...`, and both `check_model.*` methods carry the defaults.

The `bmf2bf` generic converts a `bmmformula` to a `brmsformula`. Most models use the default `bmf2bf.bmmmodel` method, but some models require custom methods to handle response variable wrapping (e.g., `bmf2bf.ddm` wraps the response as `rt | dec(response) ~ 1`) or multiple response variables (e.g., `bmf2bf.ezdm_3par` packs variables via `vreal()`, `vint()`, `trials()`). Currently, model-specific `bmf2bf` methods exist for: `ddm`, `cswald`, `ezdm_3par`, `ezdm_4par`, and `m3`.

The `configure_model` method must return a named list via `nlist()`. The required and optional fields are:

- **formula** (required) — the `brmsformula` object, with `formula$family` set to the appropriate family
- **data** (required) — the data frame (possibly with additional columns added during configuration)
- **stanvars** (optional) — custom Stan code, needed only for models with `brms::custom_family()`. Models using built-in brms families (e.g., `brms::mixture()` for the IMM) do not need `stanvars`.

Initial values are **not** returned by `configure_model` — they are handled by `create_initfun()` in a separate pipeline step.

3.6 Data flow: the attribute bridge pattern

A key architectural pattern is how information flows between `check_data()` and `configure_model()`. Since these are separate S3 methods, they cannot share local variables. Instead, `check_data` stores computed values as **attributes on the data frame**, and `configure_model` retrieves them.

```
# In check_data.non_targets (R/helpers-data.R):
attr(data, "max_set_size") <- max_set_size
attr(data, "lure_idx_vars") <- lure_idx_vars

# Later, in configure_model.imm_full (R/model_imm.R):
max_set_size <- attr(data, "max_set_size")
lure_idx <- attr(data, "lure_idx_vars")
```

This is the way to pass model-specific metadata between pipeline steps. If your `check_data` method computes something that `configure_model` needs (e.g., the maximum set size, index variables, or transformed columns), store it as a data attribute.

Two qualifications. First, the infrastructure uses the same channel for its own bookkeeping: `check_data.bmmmodel` sets `data_name` and `checked` on the data frame (R/helpers-data.R:51–52), so do not assume the attributes you find there are all yours. Second, an attribute is not a guarantee. A `configure_model` method can be called directly, bypassing `check_data` entirely, and then the attribute is absent. `configure_model.sdm` handles this by recomputing rather than failing:

```
run_metadata <- attr(data, "sdm_run_metadata")
if (is.null(run_metadata)) {
  # Guard direct configure_model.sdm() calls that bypass check_data.sdm().
  run_metadata <- sdm_run_metadata(data, formula, model)
}
```

Whether to guard this way is a judgement call. It costs a little duplicated computation and it means the expensive part has to live in a function both methods can call, which is good structure anyway. If you do not guard, make the failure loud rather than silent.

3.7 Default priors structure

The `default_priors` field in model definitions has a specific structure that the `configure_prior()` method interprets:

```
default_priors = list(
  kappa = list(
    main = "student_t(5, 1.75, 0.75)", # prior for Intercept
    effects = "normal(0, 1)",          # prior for regression coefficients
    sd = "exponential(1)"              # prior for random effect SDs (optional)
  ),
  c = list(
    main = "student_t(5, 2, 0.75)",
    effects = "normal(0, 1)"
  )
)
```

- **main:** Applied to the Intercept of the parameter. If the user suppresses the intercept (e.g., `kappa ~ 0 + condition`), this prior is applied to each level of the factor instead.
- **effects:** Applied to regression coefficients (slopes). If omitted, brms default priors are used for effects.
- **sd:** Applied to all random effect standard deviations for this parameter. This is optional — if omitted, brms default priors apply.

The priors use the same string format as `brms::set_prior()`. These defaults are automatically applied by `bmm()` unless the user explicitly overrides them via the `prior` argument.

For parameters listed in `fixed_parameters` that also appear in `parameters` (e.g., `mu1` in the IMM, which is fixed by default but can be estimated), the fixed value takes precedence unless the user provides a formula for that parameter.

3.8 Suppressing `mu` for models without a location parameter

The `brms` package requires every custom family to have a `mu` parameter (the location parameter). However, some models – particularly response time and decision models like DDM, EZDM, and `cswald` – do not have a natural location/`mu` parameter. These models suppress `mu` rather than arbitrarily promoting one of their parameters to it:

1. A dummy `mu` parameter is included in the family's `dpars` with an `identity` link
2. The `mu` intercept is fixed to 0 by adding `mu = 0` to `fixed_parameters`
3. Users do not specify a formula for `mu` in their `bmmformula`

4. The model's `bmf2bf` method constructs the first formula line differently (e.g., `rt | dec(response) ~ 1` for DDM, rather than using `mu` as a response)

This is purely a technical workaround to satisfy brms requirements; the fixed `mu = 0` ensures it has no effect on estimation.

Note

Earlier versions tracked this with a `void_mu` flag in the model definition. The flag was never read by the infrastructure – suppression is handled entirely by `fixed_parameters` and the family's `dpars` – and it has been removed. If you see `void_mu` in older code or notes, it is dead and can be deleted.

3.9 The `init_ranges` field and `create_initfun()`

Models with custom families often require carefully chosen initial values for stable MCMC sampling. When a model's likelihood has hard boundaries or regions of zero density, the default brms initialization – which draws from a wide symmetric range – will often place chains in impossible territory, causing immediate divergences or chain failures.

This is particularly relevant for response time models, where parameters have hard constraints relative to the observed data. For example, the non-decision time parameter `ndt` must be positive *and* smaller than the fastest observed RT – otherwise the implied decision time would be negative and the likelihood undefined. Similarly, boundary separation (`bound`) must be strictly positive and within a plausible range, and extreme drift rate values can cause numerical overflow in the Wiener likelihood. Circular models (`mixture2p`, `mixture3p`) typically do not have these issues and work fine with default initialization.

The `init_ranges` field in the model definition addresses this by specifying narrow, safe windows on the native (pre-link-transform) scale:

```
init_ranges = list(  
  mu      = c(-0.1, 0.1),  
  drift   = c(-1, 1),  
  bound   = c(1, 2),  
  ndt     = c(0.01, 0.05)  
)
```

Each entry maps a parameter name to a `c(lower, upper)` range. The `create_initfun()` S3 generic (defined in `R/helpers-inits.R`) uses these ranges to generate an initialization function that:

1. Parses the Stan code to determine parameter dimensions and types
2. Generates initial values for intercepts within the specified ranges (applying link transformations)
3. Generates small random values for regression coefficients, random effect SDs, and correlation matrices

If `init_ranges` is `NULL` (the default), `create_initfun` returns 1, which tells brms to use its default initialization. Most circular models (`mixture2p`, `mixture3p`) do not need custom initialization, but RT models (`DDM`, `EZDM`, `cswald`) and `SDM` do.

The conversion between native and sampling scales is handled by `link_transform()` (defined in `R/helpers-parameters.R`). It supports all link functions used by bmm models (`identity`, `log`, `softplus`, `log1p`, `logm1`, `inverse`, `sqrt`, `logit`, `probit`, `tan_half`, `cloglog`) and can apply transformations in either direction – from the scale on which parameters are interpretable (native) to the scale on which Stan samples (sampling), and back. Model developers rarely need to call it directly; it is used automatically by `create_initfun()` when generating initial values, and by the post-processing infrastructure when converting estimates back to the native scale.

3.10 Within-chain threading

Passing `threads` to `bmm()` asks brms to split each chain’s likelihood evaluation across cores. For most models nothing is needed: brms slices the data and calls the family’s likelihood inside `partial_log_lik`, and a custom family written the usual way comes along for the ride. A model that injects Stan code through `stanvar()` is the exception, because that code has to be written differently depending on whether it ends up inside `partial_log_lik` or in the model block.

`SDM` is the one model in 1.3.2 that does this (#374, up to roughly 45% faster in the release benchmarks). It is worth reading if your model has a likelihood with a normalizing constant computed over groups of rows, because the same three pieces will apply.

1. Two versions of the chunk. `inst/stan_chunks/` holds both `sdm_simple_likelihood.stan` and `sdm_simple_likelihood_threaded.stan`. The threaded one calls a `_slice` variant of the same function and passes `start` and `end`, the row bounds brms makes available inside `partial_log_lik`:

```
// serial
target += sdm_simple_run_ldenom(c, kappa, COSN, G_sdm_runs,
                                sdm_run_start, sdm_run_count);

// threaded
target += sdm_simple_run_ldenom_slice(c, kappa, COSN, start, end,
```

```
G_sdm_runs, sdm_run_start,  
sdm_run_count);
```

2. A predicate that decides which one to read. `configure_model.sdm` selects the file at configuration time:

```
likelihood_file <- if (sdm_use_threaded_likelihood()) {  
  "sdm_simple_likelihood_threaded.stan"  
} else {  
  "sdm_simple_likelihood.stan"  
}
```

The predicate reads the `brms.threads` option that `local_brms_threads()` set at the top of `bmm()` (R/model_sdm.R:234-246). It has to normalize the option the same way `brms` does, because `brms` accepts a bare number where `brms::threading()` is expected; if it did not, `brm()` would thread while `bmm` emitted the serial chunk.

3. `threading(force = TRUE)` is a separate case. With `force = TRUE`, `brms` compiles with threading support but emits *unsliced* code — there is no `partial_log_lik`, so `start` and `end` do not exist and the sliced chunk fails to compile. The predicate therefore ends with `&&!isTRUE(threads$force)`, and SDM emits its serial likelihood in that case. This was a bug fix in 1.3.2; if you write a threaded chunk, test `threading(2, force = TRUE)` explicitly, because it is not the case you will think to try.

Data that the threaded function needs must also be declared for the partial-log-likelihood signature, via the `pll_args` argument to `stanvar()`. SDM passes "data matrix COSN" for its precomputed cosine matrix and, through the helper `sdm_stanvar_int_array()`, "data array[] int <name>" for the run-index arrays. A variable that is not declared in `pll_args` is simply not in scope inside `partial_log_lik`, and the model will not compile.

3.11 Version tables

Some models support multiple versions with different parameter sets. Two patterns are used:

If/else pattern (older, used by IMM): Version-specific parameters are selected via conditional logic within the model constructor.

Version table pattern (newer, used by EZDM, cswald and M3): Version-specific parameters, links, priors, and `init_ranges` are stored in a lookup table:

```
.ezdm_version_table <- list(
  "3par" = list(
    parameters = list(...),
    links = list(...),
    fixed_parameters = list(...),
    priors = list(...),
    init_ranges = list(...)
  ),
  "4par" = list(
    parameters = list(...),
    links = list(...),
    ...
  )
)
```

The model constructor then indexes into this table for each field, selecting the entry for the requested version:

```
parameters = .ezdm_version_table[[version]][["parameters"]],
links = .ezdm_version_table[[version]][["links"]],
...
```

The table is the single source of truth for the version-specific specification. This pattern is cleaner and easier to extend than if/else logic, and new models with multiple versions should prefer it — `use_model_template("mymodel", versions = c("v1", "v2"))` generates it for you.

M3 nests the table one level deeper

The shape above is not universal, and M3 is the exception you will hit if you build a model whose parameterization depends on something other than the version. In `.m3_version_table` (R/model_m3.R:6-49), `links` and `priors` are indexed by the **choice** rule before the parameter:

```
links = list(
  simple = list(c = "log", a = "log"),
  softmax = list(c = "identity", a = "identity")
),
priors = list(
  simple = list(a = list(...), c = list(...)),
  softmax = list(a = list(...), c = list(...))
)
```

so the constructor indexes twice: `.m3_version_table[[version]][["links"]][[choice_rule]]` (R/model_m3.R:81-82). Three further departures in the same constructor are worth knowing before you copy it as a template:

- `fixed_parameters` is not in the table at all. It is computed inline from the choice rule: `list(b = if (choice_rule == "softmax") 0 else 0.1) (:76-78)`.
- `parameters` is the table entry with a background-activation parameter `b` prepended in the constructor, because every M3 version has it (:73-75).
- The table holds no `init_ranges`, and it has entries for `ss` and `cs` only. M3's default version is "custom", which is not in the table — the indexing returns NULL and the user supplies the specification. That is deliberate, not an omission.

If your model needs a second axis of variation, this is what it looks like. If it does not, keep the single-level shape the template generates.

Not every model uses either pattern

Four of the eight released models — `sdm`, `mixture2p`, `mixture3p` and `imm` — carry no `.{model}_defaults` object and no version table. They write `parameters`, `links`, `fixed_parameters`, `default_priors` and `init_ranges` as literal lists inside the constructor (see R/model_sdm.R:5-44). The separated specification is the pattern for new models and the one `use_model_template()` generates; the inline models are simply older. Do not conclude the book is wrong when you open R/model_sdm.R and find no defaults list.

3.12 File organization

The `bmm` package is organized into several files. The main files are:

R/bmm.R

It contains the main function for fitting models, `bmm()`. This function is the main entry point for users to fit models. It is set-up to be independent of the specific models that are implemented in the package.

To add new models, you do not have to edit this file. The functions above are generic S3 methods, and they will automatically recognize new models if you add appropriate methods for them (see section Adding new models).

R/helpers-*.R

R/helpers-data.R, R/helpers-inits.R, R/helpers-model.R, R/helpers-parameters.R, R/helpers-postprocess.R, and R/helpers-prior.R

These files define the main generic S3 methods for checking data, creating initial value functions, configuring the model, extracting parameters, post-processing the fitted model, and combining priors. They contain the default methods for these functions, which are called by `bmm()` if no specific method is defined for a model. If you want to add a new model, you will need to add specific methods for these functions for your model. *You do not need to edit these files to add a new model.*

R/helpers-data.R also contains several user-facing utility functions for response time data preparation. These follow a deliberate design philosophy: each function performs one well-defined task without combining too many features. They are composable – users can pipe them together into a workflow but also inspect intermediate results at each step:

- `flag_contaminant_rts()` – identifies likely contaminant trials by fitting a mixture of the RT distribution and a uniform contaminant process, returning per-trial posterior probabilities of contamination
- `validate_fast_guesses()` – takes flagged contaminants and tests whether fast responses show chance-level accuracy consistent with guessing, using a Bayesian test
- `ezdm_summary_stats()` – computes the summary statistics (mean RT, variance, accuracy) that the EZDM model requires as input, with options for robust estimation and contaminant handling

R/bmmformula.R

This file contains the definition of the `bmmformula` class, which is used to represent the formula for the model. It contains the `bmmformula()` function and its alias `bmf()`, which is used to create a new formula object.

In addition, it contains the definition of the `bmf2bf` S3 method that is used to convert a `bmmformula` object into a `brmsformula` object. This is necessary, as `brmsformula` objects are required to include the response variable in the first formula line. In contrast `bmmformula` objects only contain formulas predicting the parameters of a `bmmmodel`. The `bmf2bf` S3 method is used to perform this conversion and add the first formula line including the response variable in the `brmsformula` created during model configuration. Some models override this with custom methods (e.g., `bmf2bf.ddm`, `bmf2bf.ezdm_3par`).

R/model_*.R

Each model and its methods are defined in a separate file. The current model files are:

- `model_mixture2p.R` - Two-parameter mixture model
- `model_mixture3p.R` - Three-parameter mixture model
- `model_imm.R` - Interference Measurement Model (versions: full, bsc, abc)
- `model_sdm.R` - Signal Discrimination Model
- `model_ddm.R` - Diffusion Decision Model
- `model_ezdm.R` - EZ-Diffusion Model (versions: 3par, 4par)
- `model_cswald.R` - Censored-Shifted Wald Model (versions: simple, crisk)
- `model_m3.R` - Multinomial Memory Measurement Model (versions: ss, cs, custom)

Each file contains the internal model constructor, the user-facing alias, and model-specific S3 methods. Your new model will exist in a file like this. The name of the file should be `model_name_of_your_model.R`. You don't have to add this file manually - see section Adding new models.

R/distributions.R

This file contains the definition of the custom distributions used in the package. It specifies the density (**d***), random number generation (**r***), cumulative probability (**p***), and quantile (**q***) functions for each model's distribution. Currently implemented distributions include: `sdm`, `mixture2p`, `mixture3p`, `imm`, `m3`, `ddm`, `ezdm`, and `cswald`. If your model requires a custom distribution, you will need to add it to this file. These are not used during model fitting, but are used to generate data from the model and for posterior predictive checks.

R/conditional_effects.R, R/emmeans.R, R/pp_check.R

Post-processing infrastructure for fitted `bmmfit` objects:

- `conditional_effects.R` - `conditional_effects()` method for visualizing marginal effects
- `emmeans.R` - Integration with the `emmeans` package for estimated marginal means
- `pp_check.R` - Posterior predictive checking, including custom support for multinomial models

These work automatically for most models and do not need to be edited when adding a new model.

R/utils.R, R/brms-misc.R, R/restructure.R, R/summary.R, R/update.R

Various utility functions. Note that `R/brms-misc.R` is a read-only vendored copy from `brms` and should not be edited.

inst/stan_chunks/

This directory contains the Stan code chunks that are passed to the `brms::stanvar()` function. These define the custom likelihoods and helper functions for models that require custom `brms` families. Current Stan files:

- `sdm_simple_funs.stan`, `sdm_simple_tdata.stan`, `sdm_simple_likelihood.stan`, `sdm_simple_likelihood_threaded.stan` - SDM model
- `ddm_functions.stan` - DDM model (Wiener process likelihood)
- `ezdm_3par_functions.stan`, `ezdm_4par_functions.stan` - EZDM model
- `cswald_simple_functions.stan`, `cswald_crisk_functions.stan`, `cswald_helper_functions.stan` - cswald model

When writing Stan code for `bmm`, keep in mind that Stan's HMC sampler requires continuous gradients to function correctly. Non-differentiable operations – such as `fabs()` or `fmax()` – create discontinuities in the log-density surface that can cause divergent transitions or break sampling entirely. Where such operations are mathematically necessary, `bmm` uses smooth approximations instead. For example, `sqrt(x^2 + epsilon)` serves as a differentiable stand-in for `|x|`. This is a standard Stan modelling technique and should be applied whenever your model's likelihood involves non-smooth functions.

If you add a new model with a custom family, you will need to add Stan chunks to this directory. The naming convention is `{model_name}_{block}.stan` (e.g., `my_model_functions.stan`).

4 Example model file

All models in the package are defined as S3 classes and follow a strict template. This allows us to implement general methods for handling model fitting, data checking, and post-processing. Each model has an internal function that defines the model and its parameters, and a user-facing alias. Let's look at how two models are implemented - the IMM model, which uses both general class and specific model methods, but no custom stan code, and the SDM model, which depends heavily on custom stan code. If you use the `use_model_template()` function, templates for all sections below will be automatically generated for your model.

4.1 The Interference Measurement Model (IMM)

The model is defined in the file `R/model_imm.R`. Let's go through the different parts.

4.1.1 Model definition

The full IMM model is defined in the following internal model class:

```
.model_imm <-  
function(resp_error = NULL, nt_features = NULL, nt_distances = NULL,  
         set_size = NULL, regex = FALSE, version = "full", links = NULL,  
         call = NULL, ...) {  
  out <- structure(  
    list(  
      resp_vars = nlist(resp_error),  
      other_vars = nlist(nt_features, nt_distances, set_size),  
      domain = "Visual working memory",  
      task = "Continuous reproduction",  
      name = "Interference measurement model by Oberauer and Lin (2017).",  
      version = version,  
      citation = glue(  
        "Oberauer, K., & Lin, H.Y. (2017). An interference model \\  
        of visual working memory. Psychological Review, 124(1), 21-59"  
      ),  
      requirements = glue(  

```

```

    '- The response vairable should be in radians and \\
    represent the angular error relative to the target
    - The non-target features should be in radians and be \\
    centered relative to the target'
  ),
  parameters = list(
    mu1 = glue(
      "Location parameter of the von Mises distribution for memory \\
      responses (in radians). Fixed internally to 0 by default."
    ),
    kappa = "Concentration parameter of the von Mises distribution",
    a = "General activation of memory items",
    c = "Context activation",
    s = "Spatial similarity gradient"
  ),
  links = list(
    mu1 = "tan_half",
    kappa = "log",
    a = "log",
    c = "log",
    s = "log"
  ),
  fixed_parameters = list(mu1 = 0, mu2 = 0, kappa2 = -100),
  default_priors = list(
    mu1 = list(main = "student_t(1, 0, 1)"),
    kappa = list(main = "normal(2, 1)", effects = "normal(0, 1)"),
    a = list(main = "normal(0, 1)", effects = "normal(0, 1)"),
    c = list(main = "normal(0, 1)", effects = "normal(0, 1)"),
    s = list(main = "normal(0, 1)", effects = "normal(0, 1)")
  )
),
# attributes
regex = regex,
regex_vars = c('nt_features', 'nt_distances'),
class = c("bmmodel", "circular", "non_targets", "imm", paste0('imm_',version)),
call = call
)

# add version specific information
if (version == "abc") {
  out$parameters$s <- NULL
  out$links$s <- NULL

```

```

    out$default_priors$s <- NULL
    attributes(out)$regex_vars <- c('nt_features')
  } else if (version == "bsc") {
    out$parameters$a <- NULL
    out$links$a <- NULL
    out$default_priors$a <- NULL
  }

  out$links[names(links)] <- links
  out
}

```

Here is a brief explanation of the different components of the model definition:

resp_vars: a list of response variables that the model will be fitted to. These variables will be used to construct the **brmsformula** passed to **brms** together with the **bmmformula** and the **parameters** of the model. The user has to provide these variables in the data frame that is passed to the **bmm()** function

other_vars: a list of additional variables that are required for the model. This is used to check if the data contains all necessary information for fitting the model. In the example above, the IMM model requires the names of the variables specifying the non-target features relative to the target, the variables specifying the distance of the non-targets to the target, and the **set_size**. The user has to provide these variables in the data frame that is passed to the **bmm()** function

domain, task, name, citation, requirements: contains information about the model, such as the domain, task, name, citation, requirements. This information is used for generating help pages

version: if the model has multiple versions, this argument is specified by the user. Then it is used to dynamically adjust some information in the model object. In the case of the **imm** model, we have three versions - **full**, **bsc** and **abc**. As you can see at the end of the script, some parameters are deleted depending on the model version.

parameters: contains a named list of all parameters in the model that can be estimated by the user and their description. This information is used internally to check if the **bmmformula** contains linear model formulas for all model parameters, and to decide what information to include in the summary of **bmmfit** objects.

links: a named list providing the link function for each parameter. For example, **kappa** in the **imm** models has to be positive, so it is sampled on the log scale. This information is used in defining the model family and for the summary methods. If you want the user to be able to specify custom link functions, the next to last line of the script replaces the links with those provided by the user

fixed_parameters in the **imm** several parameters are fixed to constant values internally to identify the model. Only one of them, **mu1** is also part of the **parameters** block - this is the only fixed parameters that users can choose to estimate instead of leaving it fixed. **mu2** and **kappa2** cannot be freely estimated.

default_priors a list of lists for each parameter in the model. Each prior has two components: **main**, the prior that will be put on the Intercept or on each level of a factor if the intercept is suppressed; **effects**, the prior to put on the regression coefficients relative to the intercept. The priors are described as in the **set_prior** function from **brms**. This information is used by the **configure_prior()** S3 method to automatically set the default priors for the model. The priors that you put here will be used by **bmm()** unless the users chooses to overwrite them.

regex: For the **imm** models, the **nt_features** and **nt_distances** variables can be specified with regular expressions, if the user sets **regex = TRUE**

call: this automatically records how the model was called so that the call can be printed in the summary after fitting. Leave it as is.

class: is the most important part. It contains the class of the model. This is used by generic S3 methods to perform data checks and model configuration. The classes should be ordered from most general to most specific. A general class exists when the same operations can be performed on multiple models. For example, the '3p', 'imm_abc', 'imm_bsc' and 'imm_full' models all have non-targets and set_size arguments, so the same data checks can be performed on all of them, represented by the class **non_targets**. The first class should always be **bmmmodel**, which is the main class for all models. The last class should be the specific model name, in this case **imm_full**, **imm_abc** or **imm_bsc**, which is automatically constructed if a **version** argument is provided. Otherwise the last class will be just the name of the model.

4.1.2 Model alias

The model alias is a user-facing function that calls the internal model function. It is defined as follows:

```
#' @title `r .model_imm()$name`
#' @description Three versions of the `r .model_imm()$name` - the full, bsc, and abc.
#' `IMMfull()`, `IMMbsc()`, and `IMMabc()` are deprecated and will be removed in the future.
#' Please use `imm(version = 'full')`, `imm(version = 'bsc')`, or `imm(version = 'abc')` instead.
#'
#' @name imm
#' @details `r model_info(.model_imm(), components = c('domain', 'task', 'name', 'citation'))`
#' ##### Version: `full`
#' `r model_info(.model_imm(version = "full"), components = c('requirements', 'parameters', 'citations'))`
#' ##### Version: `bsc`
#' `r model_info(.model_imm(version = "bsc"), components = c('requirements', 'parameters', 'citations'))`
```

```

#' #### Version: `abc`
#' `r model_info(.model_imm(version = "abc"), components =c('requirements', 'parameters', 'f
#'
#' Additionally, all imm models have an internal parameter that is fixed to 0 to
#' allow the model to be identifiable. This parameter is not estimated and is not
#' included in the model formula. The parameter is:
#'
#' - b = "Background activation (internally fixed to 0)"
#'
#' @param resp_error The name of the variable in the provided dataset containing
#' the response error. The response Error should code the response relative to
#' the to-be-recalled target in radians. You can transform the response error
#' in degrees to radian using the `deg2rad` function.
#' @param nt_features A character vector with the names of the non-target
#' variables. The non_target variables should be in radians and be centered
#' relative to the target. Alternatively, if regex=TRUE, a regular
#' expression can be used to match the non-target feature columns in the
#' dataset.
#' @param nt_distances A vector of names of the columns containing the distances
#' of non-target items to the target item. Alternatively, if regex=TRUE, a regular
#' expression can be used to match the non-target distances columns in the
#' dataset. Only necessary for the `bsc` and `full` versions.
#' @param set_size Name of the column containing the set size variable (if
#' set_size varies) or a numeric value for the set_size, if the set_size is
#' fixed.
#' @param regex Logical. If TRUE, the `nt_features` and `nt_distances` arguments
#' are interpreted as a regular expression to match the non-target feature
#' columns in the dataset.
#' @param version Character. The version of the IMM model to use. Can be one of
#' `full`, `bsc`, or `abc`. The default is `full`.
#' @param ... used internally for testing, ignore it
#' @return An object of class `bmmmodel`
#' @keywords bmmmodel
#' @examplesIf isTRUE(Sys.getenv("BMM_EXAMPLES"))
#' # load data
#' data <- oberauer_lin_2017
#'
#' # define formula
#' ff <- bmmformula(
#'   kappa ~ 0 + set_size,
#'   c ~ 0 + set_size,
#'   a ~ 0 + set_size,

```

```

#'   s ~ 0 + set_size
#' )
#'
#' # specify the full IMM model with explicit column names for non-target features and distances
#' # by default this fits the full version of the model
#' model1 <- imm(resp_error = "dev_rad",
#'               nt_features = paste0('col_nt', 1:7),
#'               nt_distances = paste0('dist_nt', 1:7),
#'               set_size = 'set_size')
#'
#' # fit the model
#' fit <- bmm(formula = ff,
#'            data = data,
#'            model = model1,
#'            cores = 4,
#'            backend = 'cmdstanr')
#'
#' # alternatively specify the IMM model with a regular expression to match non-target features
#' # this is equivalent to the previous call, but more concise
#' model2 <- imm(resp_error = "dev_rad",
#'               nt_features = 'col_nt',
#'               nt_distances = 'dist_nt',
#'               set_size = 'set_size',
#'               regex = TRUE)
#'
#' # fit the model
#' fit <- bmm(formula = ff,
#'            data = data,
#'            model = model2,
#'            cores = 4,
#'            backend = 'cmdstanr')
#'
#' # you can also specify the `bsc` or `abc` versions of the model to fit a reduced version
#' model3 <- imm(resp_error = "dev_rad",
#'               nt_features = 'col_nt',
#'               set_size = 'set_size',
#'               regex = TRUE,
#'               version = 'abc')
#' fit <- bmm(formula = ff,
#'            data = data,
#'            model = model3,
#'            cores = 4,

```

```
#'          backend = 'cmdstanr')
#' @export
imm <- function(resp_error, nt_features, nt_distances, set_size, regex = FALSE, version = "f
  call <- match.call()
  dots <- list(...)
  if ("setsize" %in% names(dots)) {
    set_size <- dots$setsize
    warning2("The argument 'setsize' is deprecated. Please use 'set_size' instead.")
  }
  if (version == "abc") nt_distances <- NULL
  stop_missing_args()
  .model_imm(resp_error = resp_error, nt_features = nt_features,
             nt_distances = nt_distances, set_size = set_size, regex = regex,
             version = version, call = call, ...)
}
```

The details will be filled out automatically from the model definition. This does some fancy formatting to include documentation about all versions of the model in the same help file.

4.1.3 check_data() methods

Each model should have a `check_data.modelname()` method that checks if the data contains all necessary information for fitting the model. For the IMM, the `bsc` and `full` versions require a special check for the `nt_distances` variables:

```
#' @export
check_data.imm_bsc <- function(model, data, formula) {
  data <- .check_data_imm_dist(model, data, formula)
  NextMethod("check_data")
}

#' @export
check_data.imm_full <- function(model, data, formula) {
  data <- .check_data_imm_dist(model, data, formula)
  NextMethod("check_data")
}

.check_data_imm_dist <- function(model, data, formula) {
  nt_distances <- model$other_vars$nt_distances
  max_set_size <- attr(data, "max_set_size")
}
```

```

stopif(
  !isTRUE(all.equal(length(nt_distances), max_set_size - 1)),
  "The number of columns for non-target distances in the argument \\
  'nt_distances' should equal max(set_size)-1})"
)

# replace NA values with 999 so they have 0 effect through the distance formula
data[, nt_distances][is.na(data[, nt_distances])] <- 999

stopif(
  any(data[, nt_distances] < 0),
  "All non-target distances to the target need to be postive."
)
data
}

```

The IMM models share methods with the `mixture3p` model, all of which are of class `non_targets` so the `check_data.non_targets` method is defined in the general file `R/helpers-data.R`. If you are adding a new model, you should check if the data requirements are similar to any existing model and define the `check_data` method only for the methods that are unique to your model.

The `check_data.mymodel()` function should always take the arguments `model`, `data`, and `formula` and return the data with the necessary transformations. It should also call `NextMethod("check_data")` to call the `check_data` method of the more general class.

To understand what validation your model inherits, trace the `NextMethod` dispatch chain for your class. For example, the `imm_full` model with class `c("bmmodel", "circular", "non_targets", "imm", "imm_full")` dispatches as:

1. `check_data.imm_full` — validates `nt_distances` (model-specific)
2. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.imm` — (if defined, otherwise skipped)
3. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.non_targets` — validates `nt_features`, `set_size`, creates index variables, stores data attributes
4. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.circular` — validates response is in radians, wraps to $(-\pi, \pi]$
5. $\rightarrow$ `NextMethod` $\rightarrow$ `check_data.bmmodel` — generic checks (column existence for `resp_vars` and `other_vars`)

This means a new circular model with non-targets automatically gets radian validation, non-target checks, and column existence checks — you only need to write methods for your model-specific validations.

4.1.4 configure_model() methods

The `configure_model.mymodel()` method is where you specify the model formula, the family, any custom code. The method is defined as follows for the IMM model:

(we show only the IMMfull version)

```
configure_model.imm_full <- function(model, data, formula) {
  # retrieve arguments from the data check
  max_set_size <- attr(data, "max_set_size")
  lure_idx <- attr(data, "lure_idx_vars")
  nt_features <- model$other_vars$nt_features
  nt_distances <- model$other_vars$nt_distances

  formula <- .base_imm_formula(model, formula) +
    brms::nlf(theta1 ~ log(exp(c) + exp(a))) +
    brms::nlf(expS ~ exp(s))

  theta_exprs <- glue("{lure_idx} * log(exp(c-expS*{nt_distances})) + exp(a)) + (1 - {lure_idx}
  formula <- .add_imm_mixture_terms(formula, max_set_size, lure_idx, nt_features, theta_exprs)
  formula$family <- .imm_mixture_family(max_set_size)

  nlist(formula, data)
}
```

The three IMM versions differ only in the `theta` expression and in which extra non-linear terms they add, so everything they share lives in three private helpers at the top of `R/model_imm.R`:

```
.base_imm_formula <- function(model, formula) {
  bmf2bf(model, formula) +
  brms::lf(kappa2 ~ 1) +
  brms::lf(mu2 ~ 1) +
  brms::nlf(kappa1 ~ kappa)
}

.add_imm_mixture_terms <- function(formula, max_set_size, lure_idx, nt_features, theta_exprs) {
  kappa_nts <- paste0("kappa", 3:(max_set_size + 1))
  theta_nts <- paste0("theta", 3:(max_set_size + 1))
  mu_nts <- paste0("mu", 3:(max_set_size + 1))

  for (i in 1:(max_set_size - 1)) {
    formula <- formula +
      glue_nlf("{kappa_nts[i]} ~ kappa") +

```

```

      glue_nlf("{theta_nts[i]} ~ {theta_exprs[i]}") +
      glue_nlf("{mu_nts[i]} ~ {nt_features[i]}")
    }

    formula
  }

.imm_mixture_family <- function(max_set_size) {
  brms::mixture(
    brms::von_mises("tan_half"), brms::von_mises("identity"),
    nmix = c(1, max_set_size),
    order = "none"
  )
}

```

The version-specific method then builds `theta_exprs` as a glue vector — one expression per non-target position — and hands it to `.add_imm_mixture_terms()`, which loops. `imm_abc` passes `"{lure_idx} * a + (1 - {lure_idx}) * (-100)"`, `imm_full` the distance-weighted expression above. This is the pattern to copy if your model has versions that share a formula skeleton: put the skeleton in a helper prefixed with a dot, and let each version supply only what differs. Note the dot prefix, but not `.model_` — `supported_models()` scans for that prefix and would list your helper as a model.

The `configure_model` method should always take the arguments `model`, `data`, and `formula` (as a `bmmformula`) and return a named list with the formula (as a `brmsformula`) and the data. The `brmsfamily` should be stored within the formula.

Inside the `configure_model` method the `brmsformula` is generated using the `bmf2bf` function. This function converts the `bmmformula` passed to `bmm()` function into a `brmsformula` based on the information for the response variables provided in the `bmmmodel` object. There is a general method in `R/bmmformula.R` to construct the formula for all models with a single response variable.

```

# If no model specific methods exist, it will construct a base brms formula from the first r
# We do it this way because most models require just one response variable
#' @export
bmf2bf.bmmmodel <- function(model, formula = bmmformula()) {
  brms_formula <- NextMethod("bmf2bf") %||% brms::bf(glue("{model$resp_vars[[1]]} ~ 1"))
  components <- lapply(formula, function(x) if (is_nlf(x)) brms::nlf(x) else brms::lf(x))
  Reduce(`+`, components, init = brms_formula)
}

#' @export

```

```
bmf2bf.default <- function(model, formula) {
  NULL
}
```

Read the dispatch order carefully here, because it is the reverse of what the names suggest. `bmf2bf()` is called with the model object, whose class vector starts with `"bmmmodel"`, so `bmf2bf.bmmmodel` runs **first**. Its `NextMethod("bmf2bf")` walks on to the rest of the chain, and only reaches `bmf2bf.default` if no model-specific method intercepted.

That is why `bmf2bf.default` returns `NULL` rather than a formula. It means “nobody claimed the first line”. `bmf2bf.bmmmodel` sees the `NULL`, falls through the `%||%`, and builds the single-response first line `resp ~ 1` itself. A model with several response variables, or one needing `vreal()`, `vint()` or `trials()`, defines `bmf2bf.mymodel` that returns the first line it wants; `NextMethod()` then returns that instead of `NULL` and the `%||%` fallback never fires.

Once the first line exists, the rest is mechanical: each parameter formula becomes `brms::nlf()` if it is marked non-linear and `brms::lf()` otherwise, and `Reduce()` adds them all to the base formula. Whether a component is marked non-linear is decided in `assign_nl_attr()` (`R/bmmformula.R:375-382`) purely by whether one of its predictors is also a predicted parameter — which is the mechanism behind the name-clash warning described in the next chapter.

This conversion from a `bmmformula` object into a `brmsformula` object is done to avoid users having to specify complicated and long formulas or specifying all additional response information in the `brmsformula` themselves. For more detailed information on the use of additional response information in a `brmsformula` please see the `brmsformula` [documentation](#).

4.2 The Signal Discrimination Model (SDM)

The SDM model is defined in the file `R/model_sdm.R`. The SDM model differs in the configuration compared to the IMM model, as it requires custom STAN code. Let’s go through the different parts. As before, we start with the model definition.

4.2.1 Model definition

```
.model_sdm <- function(resp_error = NULL, links = NULL, version = "simple", call = NULL, ...)
  out <- structure(
    list(
      resp_vars = nlist(resp_error),
      other_vars = nlist(),
      domain = "Visual working memory",
```

```

task = "Continuous reproduction",
name = "Signal Discrimination Model (SDM) by Oberauer (2023)",
citation = glue(
  "Oberauer, K. (2023). Measurement models for visual working memory - \\
  A factorial model comparison. Psychological Review, 130(3), 841-852"
),
version = version,
requirements = glue(
  "- The response variable should be in radians and represent the angular \\
  error relative to the target"
),
parameters = list(
  mu = glue("Location parameter of the SDM distribution (in radians; \\
  by default fixed internally to 0)"),
  c = "Memory strength parameter of the SDM distribution",
  kappa = "Precision parameter of the SDM distribution"
),
links = list(
  mu = "tan_half",
  c = "log",
  kappa = "log"
),
fixed_parameters = list(mu = 0),
default_priors = list(
  mu = list(main = "student_t(1, 0, 1)"),
  kappa = list(main = "student_t(5, 1.75, 0.75)", effects = "normal(0, 1)"),
  c = list(main = "student_t(5, 2, 0.75)", effects = "normal(0, 1)")
),
init_ranges = list(
  mu = c(-0.5, 0.5),
  kappa = c(2.5, 3.5),
  c = c(4, 6)
)
),
class = c("bmmmodel", "circular", "sdm", paste0("sdm_", version)),
call = call
)
out$links[names(links)] <- links
out
}

```

Note that compared to the earlier IMM example, the SDM model now includes `init_ranges`.

This provides initial value ranges for MCMC sampling, which the `create_initfun()` step uses to generate appropriate starting values. The SDM model benefits from custom initialization because its custom Stan likelihood can be sensitive to extreme starting values.

The model definition is similar to the IMM model, but the SDM model only requires the user to specify the response error, but not additional variables such as non-target variables. The `class` is also different, as the SDM model is not a subclass of the IMM model. We'll skip the alias for the SDM model, as it is similar for every model.

4.2.2 `check_data()` methods

The SDM shares a class with other `circular` models, so most of the data checks are performed by `check_data.circular` method, defined in the general file `R/helpers-data.R`. The `sdm` however, samples much more quickly in Stan, if the data is sorted by the predictor variables, so we have the following custom data check method for the `sdm`:

```
#' @export
check_data.sdm <- function(model, data, formula) {
  # data sorted by predictors is necessary for speedy computation of normalizing constant
  data <- order_data_query(model, data, formula)
  attr(data, "sdm_run_metadata") <- sdm_run_metadata(data, formula, model)
  NextMethod("check_data")
}
```

This is a compact example of the attribute bridge. Having sorted the data, the method computes where each run of identical predictor values starts and how long it is — the information the Stan likelihood needs to compute its normalizing constant once per run rather than once per row — and hangs the result on the data frame as `sdm_run_metadata`. `configure_model.sdm` reads it back out a few pipeline steps later. The computation itself is a separate function, `sdm_run_metadata()`, and that matters: it is the reason `configure_model.sdm` can recompute the metadata when it is called directly and the attribute is absent.

One detail in that function is worth copying. `brms` drops rows with missing values in any model variable before fitting, so run boundaries computed on the full data frame would not line up with the rows Stan actually sees. `sdm_run_metadata()` therefore applies `stats::complete.cases()` over the response and predictor columns before computing the runs (`R/model_sdm.R:202-211`). Any index you precompute in `check_data` and pass through to Stan has the same problem.

4.2.3 configure_model() methods

The `configure_model` method for the SDM model is different compared to the IMM model, as it requires custom STAN code. The method is defined as follows:

```
#' @export
configure_model.sdm <- function(model, data, formula) {
  # note - c has a log link, but I've coded it manually for computational efficiency
  sdm_simple <- brms::custom_family(
    name = "sdm_simple",
    dpars = c("mu", "c", "kappa"),
    links = c("tan_half", "identity", "log"),
    lb = c(NA, NA, NA),
    ub = c(NA, NA, NA),
    type = "real", loop = FALSE,
    log_lik = log_lik_sdm_simple,
    posterior_predict = posterior_predict_sdm_simple
  )

  # prepare initial stanvars to pass to brms, model formula and priors
  sc_path <- system.file("stan_chunks", package = "bmm")
  stan_funs <- read_lines2(paste0(sc_path, "/sdm_simple_funs.stan"))
  stan_tdata <- read_lines2(paste0(sc_path, "/sdm_simple_tdata.stan"))
  likelihood_file <- if (sdm_use_threaded_likelihood()) {
    "sdm_simple_likelihood_threaded.stan"
  } else {
    "sdm_simple_likelihood.stan"
  }
  stan_likelihood <- read_lines2(paste0(sc_path, "/", likelihood_file))
  stan_tdata_pll_args <- if (sdm_use_threaded_likelihood()) {
    "data matrix COSN"
  }
  run_metadata <- attr(data, "sdm_run_metadata")
  if (is.null(run_metadata)) {
    # Guard direct configure_model.sdm() calls that bypass check_data.sdm().
    run_metadata <- sdm_run_metadata(data, formula, model)
  }
  stanvars <- brms::stanvar(scode = stan_funs, block = "functions") +
    brms::stanvar(scode = stan_tdata, block = "tdata", pll_args = stan_tdata_pll_args) +
    brms::stanvar(x = run_metadata$G_sdm_runs, name = "G_sdm_runs") +
    sdm_stanvar_int_array(run_metadata$sdm_run_start, "sdm_run_start", "G_sdm_runs") +
    sdm_stanvar_int_array(run_metadata$sdm_run_count, "sdm_run_count", "G_sdm_runs") +
    brms::stanvar(scode = stan_likelihood, block = "likelihood", position = "end")
}
```

```

# construct main brms formula from the bmm formula
formula <- bmf2bf(model, formula)
formula$family <- sdm_simple

# return the list
nlist(formula, data, stanvars)
}

```

Note that `configure_model` no longer returns `init` — initial values are now handled separately by `create_initfun()`, which uses the `init_ranges` defined in the model object.

This is the longest `configure_model` method in the package, so it is worth separating the parts that every custom-family model needs from the parts that are specific to SDM.

Every custom-family model needs the first two. The `brms::custom_family()` call declares the parameters (`dpars`), their link functions, bounds, whether the likelihood is vectorized (`loop = FALSE` here), and the R functions that implement `log_lik` and `posterior_predict`. For more information, see the [brms custom families vignette](#). And the Stan code is read from `inst/stan_chunks/` through `system.file()` — not a relative path — so that it is found once the package is installed rather than only when it is loaded from source. Each chunk becomes its own `brms::stanvar()`, tagged with the Stan block it belongs in.

The rest is SDM-specific, and all of it exists to make the normalizing constant cheap. Three things are going on:

- **Data passed to Stan, not just code.** `brms::stanvar(x = ..., name = ...)` passes an R *value* into the Stan data block. SDM passes the run count and, through the helper `sdm_stanvar_int_array()`, the run start and length arrays computed back in `check_data.sdm`. The helper exists because `stanvar()` would declare an integer array as *vector*, so it overwrites the generated `scode` with `array[G_sdm_runs] int <name>;`.
- **Two likelihood chunks.** Which one is read depends on `sdm_use_threaded_likelihood()`. See the threading section of the architecture chapter for why there are two and what `threading(force = TRUE)` does.
- **pll_args.** Data that the threaded likelihood needs must also be declared in the partial-log-likelihood signature. That is what `pll_args = "data matrix COSN"` on the `tdata` `stanvar` does, and what `sdm_stanvar_int_array()` sets for the index arrays. Without it the variable is out of scope inside `partial_log_lik` and the model does not compile.

Converting the `bmmformula` to a `brmsformula` and collecting all arguments is done entirely using the `bmf2bf` method.

4.2.4 Post-processing methods

Unlike the `imm` model, the `sdm` model requires some special post-processing because of the way the link functions are coded. These methods are applied *after* the `brmsfit` object is returned, at the very end of the `bmm()` pipeline:

```
#' @export
postprocess_brm.sdm <- function(model, fit, ...) {
  # manually set link_c to "log" since I coded it manually
  fit$family$link_c <- "log"
  fit$formula$family$link_c <- "log"
  fit
}

#' @export
revert_postprocess_brm.sdm <- function(model, fit, ...) {
  fit$family$link_c <- "identity"
  fit$formula$family$link_c <- "identity"
  fit
}
```

The `revert_postprocess_brm` method is the inverse of `postprocess_brm`. It is called internally when functions like `conditional_effects()` or `emmeans()` need to temporarily undo post-processing to work with the model in its original brms representation. If your `postprocess_brm` method modifies the fitted object (e.g., changing link functions, renaming parameters), you should also define a `revert_postprocess_brm` method that reverses those changes.

We also have a couple of special functions for custom families in brms (see the `log_likand` `posterior_predict` argument in the call to `brms::custom_familiy()`), which allow other typical tools from brms such `posterior_predict` of `bridgesampling` to work:

```
log_lik_sdm_simple <- function(i, prep) {
  mu <- brms::get_dpar(prep, "mu", i = i)
  c <- brms::get_dpar(prep, "c", i = i)
  kappa <- brms::get_dpar(prep, "kappa", i = i)
  y <- prep$data$Y[i]
  dsdm(y, mu, c, kappa, log = T)
}

posterior_predict_sdm_simple <- function(i, prep, ...) {
  mu <- brms::get_dpar(prep, "mu", i = i)
  c <- brms::get_dpar(prep, "c", i = i)
  kappa <- brms::get_dpar(prep, "kappa", i = i)
```

```
rsdm(length(mu), mu, c, kappa)
}
```

4.3 The Diffusion Decision Model (DDM)

The DDM model is defined in R/model_ddm.R.

4.3.1 Model definition

It demonstrates several patterns that differ from the circular models above: suppressing `mu` for a model with no natural location parameter, `init_ranges` for MCMC initialization, and a custom `bmf2bf` method for response variable wrapping.

The DDM uses a defaults table to store parameters, links, priors, and `init_ranges` in one place:

```
.ddm_defaults <- list(
  parameters = list(
    drift = "Drift rate = Average rate of evidence accumulation of the decision processes",
    bound = "Boundary separation = Distance between the decision boundaries that need to be r",
    ndt    = "Non-decision time = Additional time required beyond the evidence accumulation p",
    zr     = "Relative starting point = Starting point between the decision thresholds relativ",
  ),
  links = list(
    drift = "identity", bound = "log", ndt = "log", zr = "logit"
  ),
  fixed_parameters = list(
    zr = 0,
    mu = 0
  ),
  priors = list(
    drift = list(main = "cauchy(0,1)", effects = "normal(0,0.5)"),
    bound = list(main = "normal(0,0.5)", effects = "normal(0,0.5)"),
    ndt    = list(main = "normal(-1.5,0.5)", effects = "normal(0,0.3)"),
    zr     = list(main = "normal(0,0.5)", effects = "normal(0,0.3)")
  ),
  init_ranges = list(
    mu = c(-0.1, 0.1),
    drift = c(-1, 1),
    bound = c(1, 2),
```

```

    ndt      = c(0.01, 0.05),
    zr      = c(0.45, 0.55)
  )
)

.model_ddm <- function(rt = NULL, response = NULL, links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(rt, response),
      other_vars = nlist(),
      domain = "Decision Making / Response times",
      task = "Two-Alternative Force Choice RT",
      name = "Diffusion Decision Model",
      version = "NA",
      citation = glue(
        "Ratcliff, R. (1978). A theory of memory retrieval. Psychological Review, 85(2), 59-71."
      ),
      requirements = glue(
        "- The response time should be in seconds and \\
          represent the time between onset of the target stimulus until the response execution. \\
          - The response should be coded numerically: \\
          0 = lower response, 1 = upper response"
      ),
      parameters = .ddm_defaults[["parameters"]],
      links = .ddm_defaults[["links"]],
      fixed_parameters = .ddm_defaults[["fixed_parameters"]],
      default_priors = .ddm_defaults[["priors"]],
      init_ranges = .ddm_defaults[["init_ranges"]]
    ),
    class = c("bmmodel", "ddm"),
    call = call
  )
  out$links[names(links)] <- links
  out
}

```

Key differences from the circular models:

- **No natural μ :** The DDM has no location parameter. A dummy μ is included in the family's `dpars` and fixed to 0 via `fixed_parameters`, which satisfies brms' requirement that every custom family has a μ parameter.

- **Two response variables:** `resp_vars = nlist(rt, response)` — reaction time and binary choice. This requires a custom `bmf2bf` method.
- **init_ranges:** Specifies plausible starting ranges for each parameter to ensure stable MCMC sampling. The `create_initfun()` step uses these.
- **Short class chain:** `c("bmmmodel", "ddm")` — no intermediate domain class. RT models don't share as many `check_data` patterns as the circular models do.

4.3.2 User-facing alias

The DDM alias follows the standard pattern — calling `stop_missing_args()` and then the internal constructor:

```
ddm <- function(rt, response, links = NULL, ...) {
  call <- match.call()
  stop_missing_args()
  .model_ddm(rt = rt, response = response, links = links, call = call, ...)
}
```

4.3.3 bmf2bf method

Because the DDM has two response variables (`rt` and `response`), it needs a custom `bmf2bf` method to construct the first line of the brms formula with the proper response wrapping:

```
#' @export
bmf2bf.ddm <- function(model, formula) {
  rt <- model$resp_vars$rt
  response <- model$resp_vars$response
  brms::bf(paste0(rt, " | dec(", response, ") ~ 1"))
}
```

This creates a formula like `rt | dec(response) ~ 1`, where `dec()` tells brms which variable codes the binary decision (0 = lower, 1 = upper boundary).

4.3.4 configure_model method

```
#' @export
configure_model.ddm <- function(model, data, formula) {
  formula <- bmf2bf(model, formula)
}
```

```

ddm_family <- function(link_drift, link_bound, link_ndt, link_zr) {
  brms::custom_family(
    "ddm",
    dpars = c("mu", "drift", "bound", "ndt", "zr"),
    links = c("identity", link_drift, link_bound, link_ndt, link_zr),
    lb = c(NA, NA, 0.1, 0, 0),
    ub = c(NA, NA, NA, NA, 1),
    type = "real",
    vars = "dec[n]",
    loop = TRUE,
    log_lik = log_lik_ddm,
    posterior_predict = posterior_predict_ddm
  )
}

formula$family <- ddm_family(
  link_drift = model$links$drift,
  link_bound = model$links$bound,
  link_ndt = model$links$ndt,
  link_zr = model$links$zr
)

sc_path <- system.file("stan_chunks", package = "bmm")
stan_functions <- read_lines2(paste0(sc_path, "/ddm_functions.stan"))
stanvars <- brms::stanvar(scode = stan_functions, block = "functions")

nlist(formula, data, stanvars)
}

```

Note that:

- `mu` is included in `dpars` with an `identity` link and fixed to 0 (the `mu`-suppression pattern)
- `vars = "dec[n]"` passes the decision variable to the Stan likelihood
- The `log_lik` and `posterior_predict` functions use the `dddm()` and `rddm()` distribution functions from `R/distributions.R`
- `configure_model` returns `nlist(formula, data, stanvars)` without `init` — initialization is handled by `create_initfun()`

4.3.5 check_data method

The DDM has thorough data validation for RT and response variables:

```

#' @export
check_data.ddm <- function(model, data, formula) {
  rt_var <- model$resp_vars$rt
  response_var <- model$resp_vars$response

  # check variables exist
  stopif(not_in(rt_var, colnames(data)),
        "The RT variable '{rt_var}' is not present in the data.")
  stopif(not_in(response_var, colnames(data)),
        "The response variable '{response_var}' is not present in the data.")

  # remove NAs with warning
  if (any(is.na(data[, rt_var]))) {
    data <- data[!is.na(data[, rt_var]), ]
    warning2("Some values in {rt_var} were NA. These were removed.")
  }

  # validate RT: must be positive, in seconds (warn if > 10s or < 0.1s)
  stopif(any(data[, rt_var] < 0, na.rm = TRUE),
        "Some reaction times are lower than zero.")
  warnif(any(data[, rt_var] > 10, na.rm = TRUE),
        "Your data contains reaction times larger than 10 seconds...")

  # validate response: accepts integer (0/1), logical, or character ("upper"/"lower")
  # coerces all formats to 0/1 integer
  # ... (full validation with type coercion)

  NextMethod("check_data")
}

```

The DDM's `check_data` is a good example of thorough validation: it checks variable existence, removes NAs with warnings, validates RT ranges, and flexibly accepts multiple response coding formats (integer, logical, character) while coercing them all to 0/1.

4.4 The EZ-Diffusion Model (EZDM)

The EZDM model (R/`model_ezdm.R`) demonstrates two additional patterns: the **version table** pattern for managing multiple model versions, and working with **aggregate-level data** (summary statistics rather than trial-level observations).

4.4.1 Version table

Instead of using if/else logic to handle version-specific parameters (as the IMM does), the EZDM uses a lookup table:

```
.ezdm_version_table <- list(
  "3par" = list(
    parameters = list(
      drift = "Drift rate = ...",
      bound = "Boundary separation = ...",
      ndt = "Non-decision time = ...",
      s = "The diffusion constant = ..."
    ),
    links = list(drift = "identity", bound = "log", ndt = "log", s = "log"),
    fixed_parameters = list(s = 0, mu = 0),
    priors = list(
      drift = list(main = "cauchy(0,1)", effects = "normal(0,0.5)"),
      bound = list(main = "normal(0,0.5)", effects = "normal(0,0.5)"),
      ndt = list(main = "normal(-1.5,0.5)", effects = "normal(0,0.3)"),
      s = list(main = "normal(0,1)", effects = "normal(0,0.3)")
    ),
    init_ranges = list(
      mu = c(0, 1), drift = c(-1, 1), bound = c(1, 2),
      ndt = c(0.25, 0.5), s = c(0.99, 1.01)
    )
  ),
  "4par" = list(
    parameters = list(...), # adds zr parameter
    links = list(...),      # adds zr = "logit"
    ...
  )
)
```

The model constructor then indexes into this table:

```
.model_ezdm <- function(mean_rt = NULL, var_rt = NULL, n_upper = NULL,
                        n_trials = NULL, version = "3par", links = NULL,
                        call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(mean_rt, var_rt, n_upper),
      other_vars = nlist(n_trials),

```

```

...
parameters = .ezdm_version_table[[version]][["parameters"]],
links = .ezdm_version_table[[version]][["links"]],
fixed_parameters = .ezdm_version_table[[version]][["fixed_parameters"]],
default_priors = .ezdm_version_table[[version]][["priors"]],
init_ranges = .ezdm_version_table[[version]][["init_ranges"]]
),
class = c("bmmodel", "ezdm"),
call = call
)
if (!is.null(version)) class(out) <- c(class(out), paste0("ezdm_", version))
...
}

```

This pattern is cleaner and easier to extend than conditional logic. New models with multiple versions should prefer this approach.

4.4.2 bmf2bf for aggregate data

The EZDM operates on aggregate statistics (mean RT, variance of RT, number of upper responses, total trials) rather than trial-level data. This requires version-specific `bmf2bf` methods that pack multiple response variables into brms' special response terms:

```

#' @export
bmf2bf.ezdm_3par <- function(model, formula) {
  mean_rt <- model$resp_vars$mean_rt
  var_rt <- model$resp_vars$var_rt
  n_upper <- model$resp_vars$n_upper
  n_trials <- model$other_vars$n_trials

  brms::bf(paste0(mean_rt, " | vreal(", var_rt, ") + vint(", n_upper,
    ") + trials(", n_trials, ") ~ 1"))
}

```

This creates a formula like `mean_rt | vreal(var_rt) + vint(n_upper) + trials(n_trials) ~ 1`, where:

- `vreal()` passes real-valued additional response information to Stan
- `vint()` passes integer-valued additional response information
- `trials()` passes the trial count

These special brms terms make the additional variables available in the Stan likelihood via `vreal1[n]`, `vint1[n]`, and `trials[n]`.

4.4.3 Version-specific `configure_model`

Each EZDM version has its own `configure_model` method because the Stan likelihood functions differ:

```
#' @export
configure_model.ezdm_3par <- function(model, data, formula) {
  formula <- bmf2bf(model, formula)
  links <- model$links

  formula$family <- brms::custom_family(
    "ezdm_3par",
    dpars = c("mu", "drift", "bound", "ndt", "s"),
    links = c("identity", links$drift, links$bound, links$ndt, links$s),
    lb = c(NA, NA, 0, 0, 0),
    ub = c(NA, NA, NA, NA, NA),
    type = "real",
    log_lik = log_lik_ezdm_3par,
    posterior_predict = posterior_predict_ezdm_3par,
    loop = TRUE,
    vars = c("vreal1[n]", "vint1[n]", "trials[n]")
  )

  sc_path <- system.file("stan_chunks", package = "bmm")
  stan_functions <- read_lines2(paste0(sc_path, "/ezdm_3par_functions.stan"))
  stanvars <- brms::stanvar(scode = stan_functions, block = "functions")

  nlist(formula, data, stanvars)
}
```

Note the `vars` argument in `custom_family` — it lists the additional response terms that the Stan likelihood needs access to.

We will now look at how to construct all these parts for a new model. **Hint:** you don't have to do it manually, you can use the `use_model_template()` function to generate templates for your model.

5 Adding a new model

If you have read [Section 3](#) and [Section 4](#), you should have a pretty good idea of how the `bmm` package functions. Now it's time to add your new model.

The good news are, you don't have to add any of the files manually. The `bmm` package includes a function `use_model_template()` that generates all the files with templates for all necessary functions. Thus, you can focus on the important work of filling these templates with the relevant information without worrying about missing something critical.

What `use_model_template()` does

Run it from the root of the package. It writes `R/model_<model_name>.R` containing the parameter specification, the model constructor, the user-facing alias, and stub methods for `check_data`, `bmf2bf`, `configure_model` and `postprocess_brm`. With `custom_family = TRUE` it also creates one empty `inst/stan_chunks/<model_name>_<block>.stan` file per entry in `stanvar_blocks`, and wires the code that reads them into `configure_model`.

The full argument list is on the [reference page for `use\_model\_template\(\)`](#), which is generated from the function itself and is therefore never out of date. Four things about it are worth knowing before you call it:

- **versions is the second positional argument.** `use_model_template("gcm", TRUE)` means `versions = TRUE`, not `custom_family = TRUE`. Name the arguments.
- **The stan files are created before the R file is written.** If the call fails partway through, empty `.stan` files are already on disk and you have to remove them by hand before you can call the function again.
- **An unversioned model gets `version = "NA"`** — the literal string, not `NA`. That is what `print()` and the generated documentation will show unless you pass `versions`.
- **The template does not generate everything you may need.** There are no stubs for `check_formula()`, `configure_prior()` or `create_initfun()`; add those yourself only if your model needs them. The generated alias also omits the `@keywords bmmmodel` tag, which the package's pkgdown test expects on a user-facing model alias — add it (see [Chapter 6](#)).

5.1 Example

Let's add a new model called `gcm`. Let's assume that you have tested the model in Stan and you have the Stan code ready. We want to define a custom family for the `gcm` model, and we want to define the following blocks: `likelihood`, `functions` (see `?brms::stanvar` for an explanation of the blocks).

First you set up your system and git environment as described in Section 1. Then you can run the following code from RStudio in the root directory of the `bmm` package:

```
use_model_template("gcm", custom_family = TRUE, stanvar_blocks = c("likelihood", "functions"))
```

This will create the file `model_gcm.R` in the `R/` directory and the files `gcm_likelihood.stan` and `gcm_functions.stan` in the `inst/stan_chunks/` directory. The function will also open the files in RStudio. You will see the following output in the console:

- Modify '`inst/stan_chunks/gcm_likelihood.stan`'
- Modify '`inst/stan_chunks/gcm_functions.stan`'
- Modify '`R/model_gcm.R`'

Now you can fill the files with the appropriate code. The Stan files will be empty, but the R file will have the following structure:

```
#####!  
# MODELS #####  
#####!  
# see 'R/model_ddm.R' (flat defaults) or 'R/model_cswald.R' (versioned) for examples  
  
.gcm_defaults <- list(  
  parameters = list(  
    par1 = "Parameter 1 = description of parameter 1",  
    par2 = "Parameter 2 = description of parameter 2"  
  ),  
  links = list(  
    par1 = "identity",  
    par2 = "log"  
  ),  
  fixed_parameters = list(  
    mu = 0  
  ),  
  priors = list(  
    par1 = list(main = "normal(0, 1)", effects = "normal(0, 0.5)"),
```

```

    par2 = list(main = "normal(0, 0.5)", effects = "normal(0, 0.5)")
  ),
  init_ranges = list(
    par1 = c(-1, 1),
    par2 = c(0.5, 1.5)
  )
)

.model_gcm <- function(resp_var1 = NULL, required_arg1 = NULL, required_arg2 = NULL,
                      links = NULL, call = NULL, ...) {
  out <- structure(
    list(
      resp_vars = nlist(resp_var1),
      other_vars = nlist(required_arg1, required_arg2),
      domain = "",
      task = "",
      name = "",
      citation = "",
      version = "NA",
      requirements = "",
      parameters = .gcm_defaults[["parameters"]],
      links = .gcm_defaults[["links"]],
      fixed_parameters = .gcm_defaults[["fixed_parameters"]],
      default_priors = .gcm_defaults[["priors"]],
      init_ranges = .gcm_defaults[["init_ranges"]]
    ),
    class = c("bmmodel", "gcm"),
    call = call
  )
  out$links[names(links)] <- links
  out
}

# user facing alias
# information in the title and details sections will be filled in
# automatically based on the information in the .model_gcm()$info

#' @title `r .model_gcm()$name`
#' @name gcm
#' @details `r model_info(.model_gcm())`
#' @param resp_var1 A description of the response variable
#' @param required_arg1 A description of the required argument

```

```

#' @param required_arg2 A description of the required argument
#' @param links A list of links for the model parameters.
#' @param ... used internally for testing, ignore it
#' @return An object of class `bmmmodel`
#' @export
#' @examples
#' \dontrun{
#' # put a full example here (see 'R/model_ddm.R' for an example)
#' }
gcm <- function(resp_var1, required_arg1, required_arg2, links = NULL, ...) {
  call <- match.call()
  stop_missing_args()
  # uncomment if your model requires a specific backend:
  # stopif(!requireNamespace("cmdstanr", quietly = TRUE),
  #       'The "cmdstanr" package is required for this model.')
  .model_gcm(resp_var1 = resp_var1, required_arg1 = required_arg1, required_arg2 = required_arg2,
             links = links, call = call, ...)
}

#####!
# CHECK_DATA S3 methods #####
#####!
# A check_data.* function should be defined for each class of the model.
# If a model shares methods with other models, the shared methods should be
# defined in helpers-data.R. Put here only the methods that are specific to
# the model. See ?check_data for details.
# (YOU CAN DELETE THIS SECTION IF YOU DO NOT REQUIRE ADDITIONAL DATA CHECKS)

#' @export
check_data.gcm <- function(model, data, formula) {
  # retrieve required arguments
  required_arg1 <- model$other_vars$required_arg1
  required_arg2 <- model$other_vars$required_arg2

  # check the data (required)

  # compute any necessary transformations (optional)

  # save some variables as attributes of the data for later use (optional)

  NextMethod('check_data')
}

```

```
#####!
# Convert bmmformula to brmsformla methods #####
#####!
# A bmf2bf.* function should be defined if the default method for constructing
# the brmsformula from the bmmformula does not apply (e.g if aterms are required).
# The shared method for all `bmmmodels` is defined in bmmformula.R.
# See ?bmf2bf for details.
# (YOU CAN DELETE THIS SECTION IF YOUR MODEL USES A STANDARD FORMULA WITH 1 RESPONSE VARIABLE)

#' @export
bmf2bf.gcm <- function(model, formula) {
  # retrieve required response arguments
  resp_var1 <- model$resp_vars$resp_var1
  resp_var2 <- model$resp_vars$resp_arg2

  # set the base brmsformula based
  brms_formula <- brms::bf(paste0(resp_var1, " | ", vreal(resp_var2), " ~ 1"))

  # return the brms_formula to add the remaining bmmformulas to it.
  brms_formula
}

#####!
# CONFIGURE_MODEL S3 METHODS #####
#####!
# Each model should have a corresponding configure_model.* function. See
# ?configure_model for more information.

#' @export
configure_model.gcm <- function(model, data, formula) {
  # retrieve required arguments
  required_arg1 <- model$other_vars$required_arg1
  required_arg2 <- model$other_vars$required_arg2

  # retrieve arguments from the data check
  my_precomputed_var <- attr(data, 'my_precomputed_var')

  # construct brms formula from the bmm formula
  formula <- bmf2bf(model, formula)

  # construct the family & add to formula object
  gcm_family <- brms::custom_family(
```

```

    'gcm',
    dpars = c(),
    links = c(),
    lb = c(), # upper bounds for parameters
    ub = c(), # lower bounds for parameters
    type = '', # real for continuous dv, int for discrete dv
    loop = TRUE, # is the likelihood vectorized
  )
  formula$family <- gcm_family

  # prepare initial stanvars to pass to brms, model formula and priors
  sc_path <- system.file('stan_chunks', package='bmm')
  stan_likelihoood <- read_lines2(paste0(sc_path, '/gcm_likelihoood.stan'))
  stan_functions <- read_lines2(paste0(sc_path, '/gcm_functions.stan'))

  stanvars <- stanvar(scode = stan_likelihoood, block = 'likelihoood') +
    stanvar(scode = stan_functions, block = 'functions')

  # return the list
  nlist(formula, data, stanvars)
}

#####!
# POSTPROCESS METHODS #####
#####!
# A postprocess_brm.* function should be defined for the model class. See
# ?postprocess_brm for details

#' @export
postprocess_brm.gcm <- function(model, fit) {
  # any required postprocessing (if none, delete this section)
  fit
}

```

Now you have to:

1. **Fill the `.gcm_defaults` specification and the `.model_gcm` constructor.** Most of your work is in `.gcm_defaults`: specify the model's parameters (with descriptions), their link functions, which parameters are fixed (and to what value), the default priors, and the initialization ranges. It's crucial that you set default priors for every parameter, informed by knowledge in the field — see `.ddm_defaults` in `model_ddm.R` for a worked example. In `.model_gcm()`, rename the response and required arguments (or delete the extras if you have none) and fill in the metadata (`domain`, `task`, `name`, `citation`, `requirements`).

The constructor lists every field of the model object inline and reads the parameter specification from `.gcm_defaults`; its class must be `c('bmmmodel', 'gcm')`.

If your model has multiple versions with different parameter sets, generate the scaffold with `use_model_template("gcm", versions = c("v1", "v2"))`. Instead of the flat `.gcm_defaults`, this produces a `.gcm_version_table` with one entry per version; a constructor that reads each field out of it (`parameters = .gcm_version_table[[version]][["parameters"]]`, and so on) and appends `paste0("gcm_", version)` to the class vector; and an alias whose `version` argument is validated with `match.arg()`.

The released versioned models do not all validate `version` this way, so do not read one of them as the norm. `cswald()` uses `match.arg()` as the template does (R/model_cswald.R:177), `m3()` checks the string with `stopif()` (R/model_m3.R:183-186), and `ezdm()` does not validate in the constructor at all — an unknown version reaches `check_data.ezdm()` and errors there (R/model_ezdm.R:200). Use the template's `match.arg()` for a new model: it rejects the bad value where the user typed it.

2. **Adjust the user-facing alias.** Here you should only rename the required arguments and fill in the `@examples` section with a full example. Everything else will be filled in automatically based on the information in the `.model_gcm` function.
3. **Fill the `check_data.gcm` function** with the appropriate code. This function should check the data and return the data. You may or may not need to compute any transformations or save some variables as attributes of the data.
4. **If necessary, define the `bmf2bf.gcm` method** to convert the `bmmformula` to a `brmsformula`. You need a custom `bmf2bf` method if:
 - Your model has **multiple response variables** (e.g., DDM needs `rt | dec(response) ~ 1`)
 - Your model uses **aggregate data** with `vreal()`, `vint()`, `trials()` (e.g., EZDM)
 - Your model needs special response wrapping

Keep in mind that `brms` automatically interprets this formula as the linear model formula for the `mu` parameter of your custom family. If your model does not have a natural `mu` parameter, suppress it: include `mu = 0` in `fixed_parameters`, add a dummy `mu` (identity link) to the family's `dpars`, and write the first formula line yourself (as DDM does with `rt | dec(response) ~ 1`). See the DDM example in Section 4 and the *Suppressing `mu`* section of the architecture chapter for details. *If your model has a single response variable and no special wrapping, you can delete this section.*

5. **Fill the `configure_model.gcm` function** with the appropriate code. This function should construct the formula, the family, and the stanvars, and return them as `nlist(formula, data, stanvars)`. Note that `configure_model` does **not** return

initial values — initialization is handled separately by `create_initfun()` using the `init_ranges` field in your model definition.

You need to fill information about your custom family, and then fill the STAN files with your STAN code. Conveniently, loading the STAN files and setting up the stanvars is set up automatically when calling the `use_model_template` function. Should you need to add more STAN files after you created the template, you can add the files in `inst/stan_chunks/` manually and edit those lines to additionally load the manually added files.

6. **Define `init_ranges` if needed.** If your model uses a custom family (especially RT models), you should define `init_ranges` in your model definition to provide plausible starting ranges for MCMC sampling. This is particularly important for models where parameters have hard constraints relative to observed data – for example, non-decision time must be less than the fastest observed RT, otherwise the likelihood is undefined. The `create_initfun()` step will automatically use these to generate an initialization function. Each entry maps a parameter name to a `c(lower, upper)` range. If `init_ranges` is `NULL`, brms' default initialization is used.
7. **Fill the `postprocess_brm.gcm` function** with the appropriate code. By post-processing, we mean changes to the fitted brms model - like renaming parameters, etc. If you don't need any post-processing, you can delete this section.
8. **Define `log_lik` and `posterior_predict` functions** if your model uses a custom family. These R functions must be passed to `brms::custom_family()` and enable posterior predictive checks, model comparison (via LOO/WAIC), and `posterior_predict()`. They should use the `d*()` and `r*()` distribution functions from `R/distributions.R`. See the DDM example in Section 4.

5.2 Failure modes to expect

Five things in 1.3.2 will bite you while the model is still half-written. All of them are new or newly strict, so an older model file will not warn you about them.

`configure_prior()` errors if `fixed_parameters` names a parameter the formula never gets. Every name in `fixed_parameters` has to be wired into the formula by your `configure_model()`, as either a distributional parameter or a non-linear one. If it is neither, bmm stops with a model-definition error rather than letting a malformed `b_Intercept ~ constant()` prior reach `brm()` (#377, `R/helpers-prior.R:165-174`). This is the error you get when you add `mu = 0` to `fixed_parameters` but forget to put a dummy `mu` in the family's `dpars`.

Short parameter names are risky, and the rule is a word boundary. `create_initfun()` has to map Stan parameter names back to your model parameters to know which `init_ranges`

entry applies. It matches with the regex `(^|_)param(_|$)` and takes the longest match when several apply (#354, #355, R/helpers-inits.R:75-87). So a parameter called `s` matches `b_s_Intercept` but not `sd_id_sim`, which is the point — before 1.3.2 this was a substring match and `s` collided with `sim`. Two consequences for naming: a parameter name that is a substring of another is now safe, but a name that appears as a whole underscore-delimited token somewhere else in the Stan parameter names is not. If nothing matches at all, the fallback is the first underscore-delimited token of the Stan name, which is usually wrong and silently so.

A parameter that is not a dpar is looked up in nlpars. `create_initfun()` resolves each parameter's terms as `bterms$dpars[[parameter]] %||% bterms$nlpars[[parameter]]` (R/helpers-inits.R:56). Before #362 it only looked in `dpars`, and a model built as a non-linear brms formula — a native-multinomial model, for instance — failed with no applicable method for 'has_intercept' applied to an object of class "NULL". If you see that error on an older bmm, this is what it was.

A predictor that is also a parameter name and also a data column now warns. bmm decides whether a formula component is non-linear purely by whether one of its predictors is also a predicted parameter, and emits `nlf()` rather than `lf()` in that case. When the same name is also a column in the data, that choice changes the likelihood and the user has no way to see it. `check_formula.bmmmodel` warns (#378, R/bmmformula.R:165-180). Short parameter names collide naturally with column names like `accuracy` or a condition coded `c`, which is another reason to avoid one-letter parameters in a new model.

Add your model to response_annotations(). Since #392, `print.bmmmodel()` shows the required response variables with the coding they expect (“radians in `[-pi, pi]`”, “seconds”) and the default links, which answers “what does my data frame need to look like?” at the console. The annotations come from `response_annotations()` (R/helpers-model.R:184-205), a lookup keyed on the model's class. A model whose class is not in that chain falls through to an empty list and prints its response variables bare. Add a branch for your class.

5.3 Testing

Unit testing is extremely important. You should test your model with the `testthat` package. You can use the `use_test()` function to create a test file for your model. See files like `tests/testthat/test-model_ddm.R` or `tests/testthat/test-model_cswald.R` for examples of how we test models. BRMS models take a long time to fit, so we don't test the actual fitting process.

Instead we use brms' mock backend. This is not a bmm feature — `backend = "mock"` and `mock_fit` are arguments of `brms::brm()`, and they reach it because `bmm()` passes `...` through. The whole backend is two lines: whatever you pass as `mock_fit` is returned in place of the fitted Stan object (a function is called first). Everything before the fit — data checks, formula

translation, family construction, stanvars, initial values — runs for real, which is exactly what you want to test. For example, here's a mock test pattern:

```
test_that("gcm model runs without errors", {
  withr::local_options("bmm.silent" = 2)
  skip_on_cran()

  dat <- data.frame(
    resp = rnorm(10),
    exemplar = rep(c("A", "B"), 5),
    category = rep(c(1, 2), each = 5)
  )

  f <- bmmformula(param1 ~ 1, param2 ~ 1)
  mock_fit <- bmm(
    f, dat,
    gcm(resp_var1 = "resp", required_arg1 = "exemplar", required_arg2 = "category"),
    backend = "mock", mock_fit = 1, rename = FALSE
  )
  expect_equal(mock_fit$fit, 1)
})
```

Two things about that call are easy to get wrong. The alias generated by the template calls `stop_missing_args()`, so every argument without a default has to be supplied — a test that passes only the response variable errors before it reaches the pipeline. And the argument is `mock_fit`, not `mock`. You will see `mock = 1` in the existing test files (48 occurrences against 21 of `mock_fit = 1` in 1.3.2); it works only because R partial-matches it to `mock_fit` on the way through Write `mock_fit`.

The tests based on the `testthat` package are run every time you call the `check()` command. Before you submit your changes, make sure that all tests pass.

5.3.1 Parameter recovery

Additionally, you should perform a full test of the model by running it in a separate script and ensuring it gives meaningful results. Create a parameter recovery script at `local/<model_name>_parameter_recovery.R`. At the very least, you should perform basic parameter recovery simulations for hyper-parameters (i.e., means and standard deviations) as well as subject-level parameters to give users an idea of how much data they need to adequately estimate the model.

5.3.2 Distribution functions

If your model uses a custom family, you should also add distribution functions (`d*`, `r*`, and optionally `p*`, `q*`) to `R/distributions.R`. These are used for:

- Data simulation (useful for parameter recovery and examples)
- Posterior predictive checks via the `log_lik` and `posterior_predict` functions
- Plotting model fits

5.4 Post-processing infrastructure

The `bmm` package provides several post-processing tools that work automatically for most models:

- `conditional_effects()` — visualize marginal effects of predictors on model parameters. Works automatically via the `bmmfit` class.
- **emmeans integration** — estimated marginal means via the `emmeans` package. Works automatically.
- `pp_check()` — posterior predictive checks. Works automatically for standard families; custom support exists for multinomial families (like M3).

For these to work with your custom family, you need to have defined `log_lik` and `posterior_predict` functions and passed them to `brms::custom_family()`. No additional code is typically needed in your model file.

5.5 Add an example dataset

All new models should come with an example dataset, that can be loaded by users and can be used in the examples section. This should be either:

- A new dataset that you add to the package
- A dataset that already exists in the package but that can be used with the new model
- A dataset that exists in another package that you can load with `data()` and use with the new model

For example, the vignettes for the `mixture2p` and `mixture3p` use an external dataset from the `mixture` package that can be loaded with `data('bays2009_full', package='mixture')`. The `IMM` models use a dataset included in the current package. For instructions on how to add a new dataset see [here](#).

5.6 Add an article

All new models should come with an article that explains some basic information about the model and how to estimate it with `bmm`. You can use the `use_article()` function to create a new article. See [here](#) for more information. The articles will be published automatically on the package website under “Articles” when the pull request is approved. You can browse the source code for the existing articles in the `vignettes/articles/` directory. You can see the published version of the existing vignettes [here](#).

And that’s it! You have added a new model to the `bmm` package. You can now submit your changes to the `bmm` package repository.

6 Coding conventions

This chapter is the reference for how bmm code is written. It exists because your pull request will be judged against these conventions, and until now they lived only in files inside the package repository that an outside contributor has no reason to read. Everything here is checked against bmm 1.3.2.

Read it as conventions, not as a style war. Most of the entries have a reason attached, and the reason is usually that the alternative broke something.

6.1 Errors, warnings and messages

Use the package's own helpers rather than the base functions:

Instead of	Use	Why
<code>stop()</code>	<code>stop2()</code>	Interpolates with glue and sets <code>call. = FALSE</code>
<code>warning()</code>	<code>warning2()</code>	Same
<code>message()</code>	<code>message2()</code>	Same, and respects <code>options(bmm.silent)</code>
<code>if (cond)</code> <code>stop2(...)</code>	<code>stopif(cond, ...)</code>	One line, and reads as the condition it guards
<code>if (cond)</code> <code>warning2(...)</code>	<code>warnif(cond, ...)</code>	Same
<code>sprintf()</code>	<code>glue::glue()</code>	Named interpolation, no positional arguments to miscount
<code>paste(x, collapse = ", ")</code>	<code>collapse_comma(x)</code>	Also quotes each element

In 1.3.2 the counts are `stopif` 173, `stop2` 38, `warning2` 27, `warnif` 22, `message2` 11. The base equivalents appear four times, and every one of them is either inside the definition of `stop2()/warning2()` themselves (R/utils.R:169, :174) or in R/brms-misc.R, which is a vendored read-only copy of brms code.

Two of the reasons deserve spelling out, because they are not stylistic.

`message2()` checks `getOption("bmm.silent", 1)` and prints nothing at silence level 2 or above (R/utils.R:177-184). Tests set `withr::local_options("bmm.silent" = 2)` to keep output clean. A base `message()` call ignores that and will make somebody's test noisy.

`stopif()` and `warnif()` pass `env.frame = -2` down to `stop2()` (R/utils.R:189-199), which is what lets the message interpolate variables from *your* function rather than from inside the helper. So this works and needs no `glue()` call of its own:

```
stopif(
  !version %in% c("custom", "cs", "ss"),
  'Unknown version: {version}. It should be one of "ss", "cs" or "custom"'
)
```

Write messages as one glue string. `stop2()` handles multi-line strings with trailing `\\` continuations, so a long message does not need `paste0()`.

6.2 Dependencies

Package code is **base R plus the native pipe** `|>`. `bmm` requires R `>= 4.1.0`, so `|>` is always available and `%>%` is not used in package code.

DESCRIPTION Imports at 1.3.2 are `bayesplot`, `brms`, `crayon`, `fs`, `glue`, `matrixStats`, `methods`, `parallel`, `rtdists`, `rlang`, `stats` and `withr`. **dplyr, tidyr, ggplot2 and stringr are Suggests, not Imports** — 1.3.0 removed the `dplyr/tidyr/magrittr` dependencies. If you have seen `bmm` guidance that prescribes `dplyr` idiom for package code, it is out of date. Tidyverse packages belong in articles and tests, where they are already available.

A function from a **Suggests** package must be guarded, or the package breaks for anyone who installed it without that suggestion. The pattern is:

```
stopif(
  !requireNamespace("tidyr", quietly = TRUE),
  'The "tidyr" package is required for this functionality'
)
data <- tidyr::gather(data, non_target_name, non_target_value, eval(nt_features))
```

as at R/helpers-data.R:174-178. Note the same function opens with

```
y <- y_nt <- non_target_name <- non_target_value <- NULL
```

which exists to silence R CMD check’s “no visible binding for global variable” notes from the non-standard evaluation in the `gather()` call. If you write code with bare column names, you need this line.

Always namespace explicitly: `brms::stanvar()`, `stats::complete.cases()`, `methods::formalArgs()`. `bmm` imports few functions directly, and an unqualified call to an Imports package is the kind of thing that works on your machine and fails in R CMD check.

6.3 Code shape

Return the last expression; use `return()` only to leave early. The 74 `return()` calls in 1.3.2 are almost all guard clauses, and that is the endorsed use:

```
x <- try_read_bmmfit(file, file_refit)
if (!is.null(x)) {
  return(x)
}
```

Do not write `return(out)` as the last line of a function.

No `set.seed()` inside a function, and no `seed` argument. A function that sets the seed silently rewrites the caller’s random state. In 1.3.2 `set.seed()` appears six times and all six are in roxygen `@examples` blocks, where it is the right thing. If you need reproducibility inside a function, take the draws you were given; if you need it in a test, set the seed in the test.

snake_case for functions and variables. Private helpers are prefixed with a dot (`.base_imm_formula`, `.imm_mixture_family`). One reservation: a private helper must **not** be named `.model_*`, because `supported_models()` lists every object matching `^\.model_` as a model (R/helpers-model.R:251).

Comments say why, not what. `# construct the family above a brms::custom_family()` call adds nothing. `# note - c has a log link, but I've coded it manually for computational efficiency` (R/model_sdm.R:124) is worth its line, and so is every comment that records a constraint somebody would otherwise remove.

6.4 S3 conventions

Dispatch runs general to specific. A model’s class vector starts with `"bmmodel"`, so `check_data.bmmodel` runs *before* `check_data.sdm`, not after. This is the opposite of the intuition most people bring, and it has one sharp corollary: **a fall-through fallback belongs on `.default`, never on `.bmmodel`.** A method defined on `.bmmodel` runs first and shadows nothing, but a *fallback* placed there would be reached before the model-specific method had a chance

to run. `bmf2bf` is the worked example — `bmf2bf.default` returns `NULL` and `bmf2bf.bmmodel` supplies the fallback after `NextMethod()` has come back empty (see Chapter 4).

Every `check_data` method ends with `NextMethod("check_data")`. Each method does its own model-specific validation and then hands on down the chain. In 1.3.2 there are 11 `check_data` methods and 10 `NextMethod("check_data")` calls; the one without it is `check_data.default`, which is the end of the chain and returns the data. Forget the `NextMethod()` and your model silently skips every generic check, including whether its response column exists.

Match the generic’s signature exactly, including `...` where the generic has one and the defaults on `check_model(model, data = NULL, formula = NULL)`. See the generics table in Chapter 3.

6.5 Where to validate arguments

Validate in the **exported constructor only** — the user-facing alias. `.model_*`(), `configure_model.*()` and private helpers should not, because their error messages name arguments the user never typed. The alias is the one place where the argument the message refers to is the argument the user actually passed:

```
m3 <- function(resp_cats, num_options, choice_rule = "softmax", version = "custom", ...) {
  call <- match.call()
  stop_missing_args()
  stopif(
    length(num_options) != length(resp_cats),
    "The option variables should have the same length as the response variables."
  )
  .model_m3(...)
}
```

`check_data.*()` is the exception, and not really an exception: it validates properties of the *data*, which the constructor cannot see. “Your `nt_distances` columns contain negative values” is a data message and belongs there.

6.6 `log_lik` and `posterior_predict`

These are **plain functions, not S3 methods**. Do not name them `log_lik.mymodel` — that would register them against brms’ `log_lik` generic, which is not what you want. Name them `log_lik_<family_name>` and pass them to `brms::custom_family()`:

```
sdm_simple <- brms::custom_family(
  name = "sdm_simple",
  dpars = c("mu", "c", "kappa"),
  ...
  log_lik = log_lik_sdm_simple,
  posterior_predict = posterior_predict_sdm_simple
)
```

brms checks the first arguments when you build the family: `log_lik` must start with `(i, prep)` and `posterior_predict` with `(i, prep, ...)`.

The contract inside is easy to get wrong in one specific way. `brms::get_dpar(prep, "par", i = i)` returns a **vector with one value per posterior draw**, not a scalar, and your function must return a vector of that same length. The response is a scalar, `prep$data$Y[i]`:

```
log_lik_sdm_simple <- function(i, prep) {
  mu <- brms::get_dpar(prep, "mu", i = i)
  c <- brms::get_dpar(prep, "c", i = i)
  kappa <- brms::get_dpar(prep, "kappa", i = i)
  y <- prep$data$Y[i]
  dsdm(y, mu, c, kappa, log = T)
}
```

A density function that collapses its arguments (an `if` on a vector, a `sum()`) will return a scalar here, and brms will recycle it across all draws without complaint. The posterior predictive checks then look suspiciously clean. Check `length()` of what you return.

`loop` in `custom_family()` is about the generated *Stan* code, not about this R function. With `loop = FALSE` brms emits a vectorized likelihood call, and with `loop = TRUE` it emits one call per observation. The consequence you will hit: with `loop = FALSE` you may not use the `[n]` index in `vars`, and brms stops with “Invalid use of index ‘[n]’ in an unlooped custom likelihood”. SDM uses `loop = FALSE` and no `vars`; EZDM uses `loop = TRUE` with `vars = c("vreal1[n]", "vint1[n]", "trials[n]")`.

6.7 Naming traps

Each of these cost somebody real debugging time, and none of them produce an error message that points at the cause.

A custom family’s `dpars` may not contain `_` or `.`, and may not end with a digit. Both are checked in `brms::custom_family()` (brms 2.23.0): “Dots or underscores are not allowed in ‘dpars’” and “‘dpars’ should not end with a number”. So neither `delta_1` nor `delta1` is a legal

dpar name. The digits you see on IMM parameters (`kappa1`, `theta3`, `mu2`) are generated by `brms::mixture()` itself, not declared by `bmm`, and non-linear parameters introduced through `nlf()` are not subject to this check.

Reusing a brms dpar name silently inherits its default prior. `brms:::def_dpar_prior()` attaches a default prior by dpar name, and the names it recognizes are `mu`, `sigma`, `shape`, `nu`, `phi`, `kappa`, `beta`, `zi`, `hu`, `zoi`, `coi`, `bs`, `ndt`, `bias`, `quantile`, `xi`, `alpha`, `disc` and `theta`. Worse, the lookup goes through `brms:::dpar_class()`, which strips trailing digits, so `kappa1` is looked up as `kappa`. `bmm` reuses several of these names deliberately and sets its own defaults over the top, which is fine — the trap is a parameter you reuse and then *forget* to give a default prior, which will quietly get `brms'` prior for an unrelated quantity.

A dpar must not be uniquely prefixed by thres. `brms:::stan_log_lik_custom()` assigns `p$thres` only for ordinal families but reads `p$thres` unconditionally when it builds the likelihood call. `p` is a list keyed by dpar name, and `$` partial-matches on lists, so a dpar named `threshold` is found by `p$thres` and passed where the ordinal thresholds belong. The result is Stan code that does not parse, from a name that looks perfectly reasonable.

Stan probability-function suffixes require the bar syntax at every call site. `stanc` 2.39 says it plainly:

Probability functions with suffixes `_lpdf`, `_lupdf`, `_lpmf`, `_lupmf`, `_cdf`, `_lcdf` and `_lccdf`, require a vertical bar (`|`) between the first two arguments.

A helper you name `sdt_cdf` therefore cannot be called as `sdt_cdf(x, mu)`; it has to be `sdt_cdf(x | mu)`. If the function is not really a distribution function, give it a different name — `sdt_cumprob` — and the problem disappears. `bmm`'s own distribution functions take the other route and use the bar: `swald_lpdf(rt | drift, bound, ndt, s)` and `swald_lccdf(rt | -drift, bound_lower, ndt, s)` in `inst/stan_chunks/cswald*.stan`.

An nlf() right-hand side is pasted together as text, so operator precedence is yours to manage. The IMM builds its `theta` expressions with glue and parenthesizes every sub-expression, including the constant: `"{lure_idx} * log(exp(c-expS*{nt_distances}) + exp(a)) + (1 - {lure_idx}) * (-100)"` (`R/model_imm.R:328`). When you interpolate an expression into a larger one, wrap it.

6.8 Documentation and the checks that enforce it

`NAMESPACE` and everything in `man/` are **generated**. Run `devtools::document()`; never hand-edit them.

Exported functions need `@returns` and `@examples`. Two further requirements are enforced by tests rather than by review:

@keywords. `tests/testthat/test-pkgdown.R` reads every `man/*.Rd` file, drops the ones with `\keyword{internal}`, and fails if any remaining topic is not covered by the reference section of `_pkgdown.yml`. Coverage can be by name or through a `has_keyword()` selector, and `_pkgdown.yml:56` has `has_keyword("bmmmodel")`. So a new model's user-facing alias needs `#' @keywords bmmmodel`, which all eight released model aliases carry and which `use_model_template()` does **not** generate. Add it yourself. An exported function that is not a model gets `@keywords internal` or a place in `_pkgdown.yml`.

A **NEWS** entry for every user-visible change, under the right heading in `NEWS.md`, with the issue or PR number in parentheses. Read the 1.3.2 section for the expected level of detail: entries say what changed, what the old behaviour was, and why it mattered, not just “fixed a bug”.

6.9 Tests

Test files mirror source file names — `R/model_ddm.R` is tested by `tests/testthat/test-model_ddm.R`. This is a convention rather than a rule: at 1.3.2 nine files under `R/` have no matching test file and six test files have no matching source file, usually because they test a theme (`test-default-priors.R`, `test-pkgdown.R`) rather than a file. A new model gets a matching test file.

Use the mock backend for pipeline tests and `testthat` edition 3 (`Config/testthat/edition:3` in `DESCRIPTION`). Set `withr::local_options("bmm.silent" = 2)` so `message2()` output does not clutter the run.

6.9.1 Parameter recovery, and the circularity trap

A new model needs a parameter recovery script at `local/<model_name>_parameter_recovery.R`. `local/` is untracked, so the script is not run by `R CMD check` and does not need to be fast. It is expected for every new model, and it is what the reviewer who installs your branch will look at.

Do not simulate and fit with the same distribution function. If you generate data with your own `rmymodel()` and then fit a likelihood built from your own `dmymodel()`, the study will report clean recovery whether or not either function is correct — any error shared by both cancels. Use an independent generator: `rtdists::rdiffusion()` for diffusion-type models (`rtdists` is already an Imports), a published reference implementation, or a direct simulation of the process the model describes. A recovery study that cannot fail is not evidence.

Recover both the hyper-parameters (means and standard deviations) and the subject-level parameters, and report how much data was needed, because that is the question a user will ask.